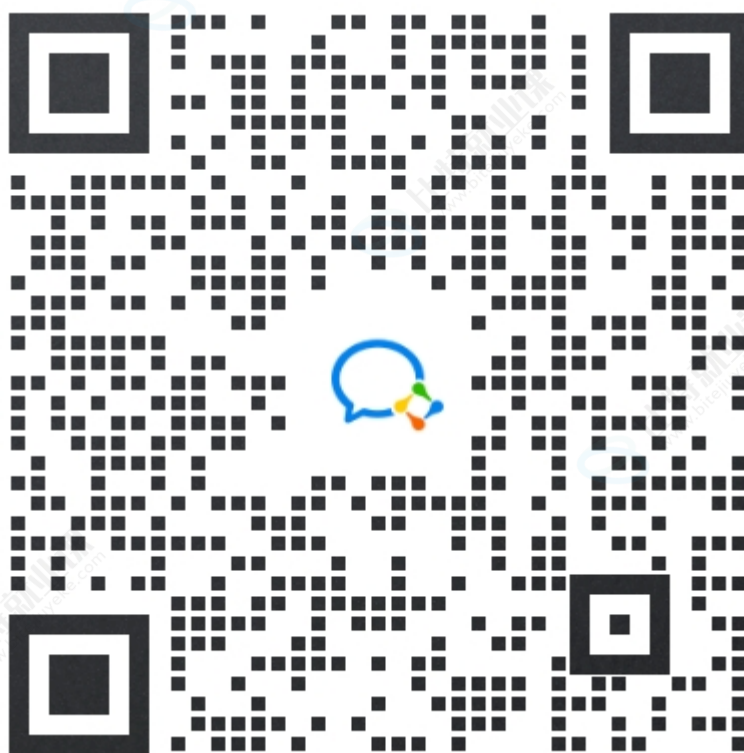


Java - 仿 RabbitMQ 的消息队列

版权说明

本“比特就业课”项目（以下简称“本项目”）的所有内容，包括但不限于文字、图片、音频、视频、软件、程序、数据库、设计、布局、界面等，均由本项目的开发者或授权方拥有版权。我们鼓励个人学习者使用本项目进行学习和研究。在遵守相关法律法规的前提下，个人学习者可以下载、浏览、学习本项目的内容，并为了个人学习、研究或教学目的而使用其中的材料。但请注意，未经我们明确授权，个人学习者不得将本项目的内容用于任何商业目的，包括但不限于销售、转让、许可或以其他方式从中获利。此外，个人学习者也不得擅自修改、复制、传播、展示、表演或制作本项目内容的衍生作品。任何未经授权的使用均属侵权行为，我们将依法追究法律责任。如果您希望以其他方式使用本项目的内容，包括但不限于引用、转载、摘录、改编等，请事先与我们联系，获取书面授权。感谢您对“比特就业课”项目的关注与支持，我们将持续努力，为您提供更好的学习体验。特此说明。比特就业课版权所有

对比特项目感兴趣，可以联系这个微信。



代码 & 板书链接

<https://gitee.com/HGtz2222/bitproject/tree/master/%E6%A8%A1%E6%8B%9F%E5%AE%9E%E7%8E%B0%E6%B6%88%E6%81%AF%E9%98%9F%E5%88%97>

一. 消息队列背景知识

曾经我们学习过 **阻塞队列 (BlockingQueue)** , 我们说, 阻塞队列最大的用途, 就是用来实现 **生产者消费者模型**.

生产者消费者模型, 存在诸多好处, 是后端开发的常用编程方式.

- 解耦合
- 削峰填谷

在实际的后端开发中, 尤其是分布式系统里, 跨主机之间使用生产者消费者模型, 也是非常普遍的需求.

因此, 我们通常会把阻塞队列, 封装成一个独立的服务器程序, 并且赋予其更丰富的功能.

这样的程序我们就称为 **消息队列 (Message Queue, MQ)**

市面上成熟的消息队列非常多.

- RabbitMQ
- Kafka
- RocketMQ
- ActiveMQ
-

其中, RabbitMQ 是一个非常知名, 功能强大, 广泛使用的消息队列.

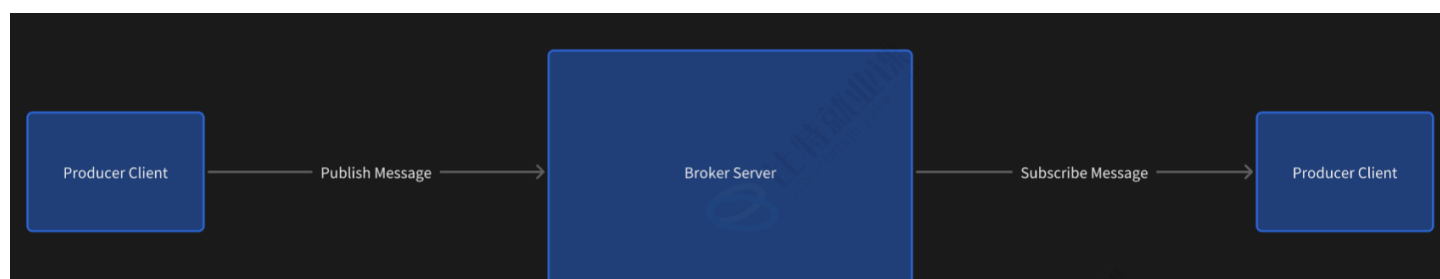
咱们就仿照 RabbitMQ, 模拟实现一个简单的消息队列.

二. 需求分析

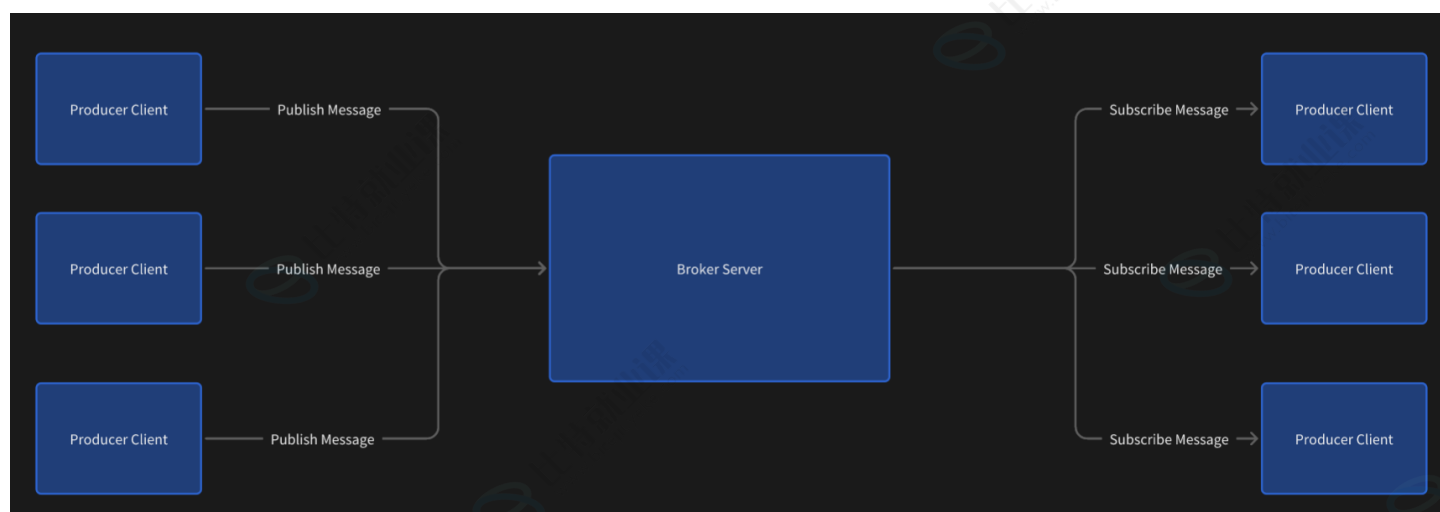
核心概念

- 生产者 (Producer)
- 消费者 (Consumer)
- 中间人 (Broker)
- 发布 (Publish)
- 订阅 (Subscribe)

一个生产者, 一个消费者



N 个生产者, N 个消费者



其中, Broker 是最核心的部分. 负责消息的存储和转发.

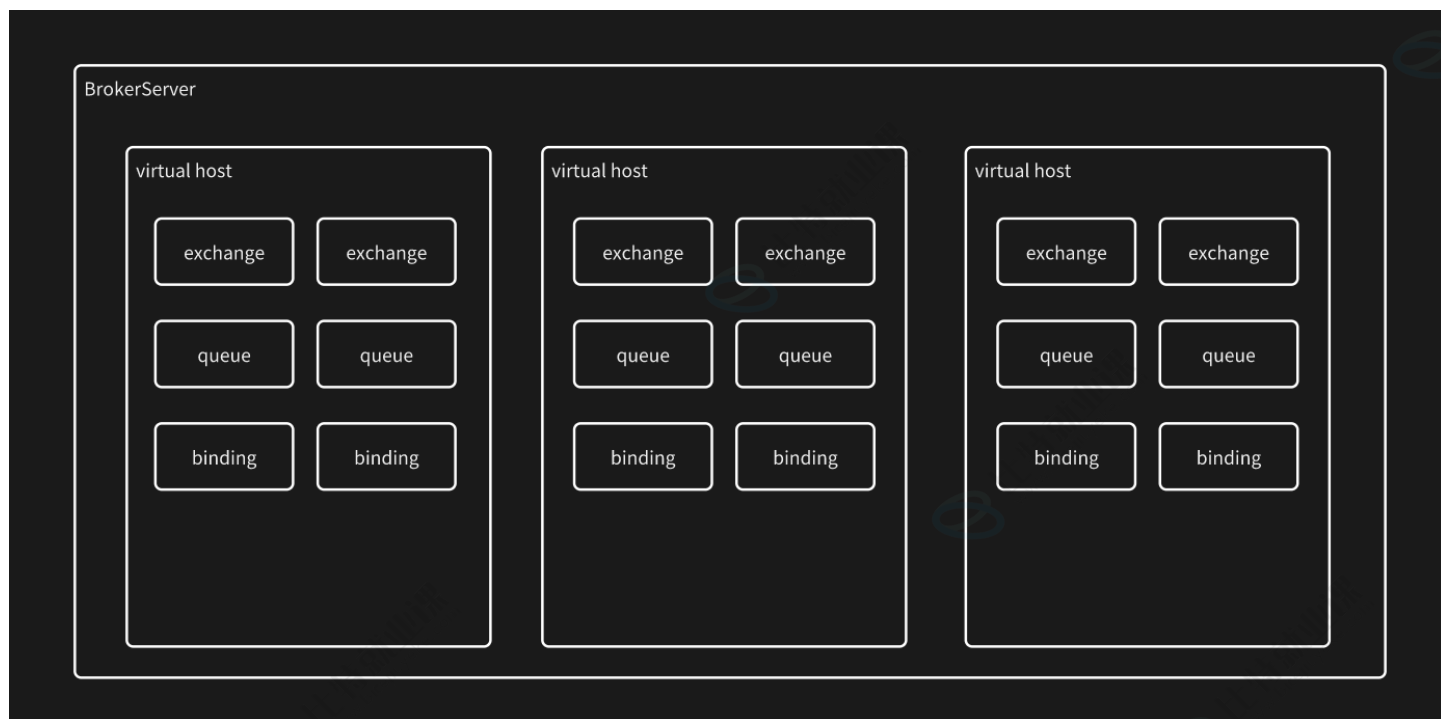
在 Broker 中, 又存在以下概念.

- 虚拟机 (VirtualHost): 类似于 MySQL 的 "database", 是一个逻辑上的集合. 一个 BrokerServer 上可以存在多个 VirtualHost.
- 交换机 (Exchange): 生产者把消息先发送到 Broker 的 Exchange 上. 再根据不同的规则, 把消息转发给不同的 Queue.
- 队列 (Queue): 真正用来存储消息的部分. 每个消费者决定自己从哪个 Queue 上读取消息.
- 绑定 (Binding): Exchange 和 Queue 之间的关联关系. Exchange 和 Queue 可以理解成 "多对多" 关系. 使用一个关联表就可以把这两个概念联系起来.
- 消息 (Message): 传递的内容.

所谓的 Exchange 和 Queue 可以理解成 "多对多" 关系, 和数据库中的 "多对多" 一样. 意思是:

一个 Exchange 可以绑定多个 Queue (可以向多个 Queue 中转发消息).

一个 Queue 也可以被多个 Exchange 绑定 (一个 Queue 中的消息可以来自于多个 Exchange).



这些概念,既需要在内存中存储,也需要在硬盘上存储.

- 内存存储: 方便使用.
- 硬盘存储: 重启数据不丢失.

核心 API

对于 Broker 来说,要实现以下核心 API. 通过这些 API 来实现消息队列的基本功能.

1. 创建队列 (queueDeclare)
2. 销毁队列 (queueDelete)
3. 创建交换机 (exchangeDeclare)
4. 销毁交换机 (exchangeDelete)
5. 创建绑定 (queueBind)
6. 解除绑定 (queueUnbind)
7. 发布消息 (basicPublish)
8. 订阅消息 (basicConsume)
9. 确认消息 (basicAck)

另一方面, Producer 和 Consumer 则通过网络的方式, 远程调用这些 API, 实现 **生产者消费者模型**.

关于 VirtualHost

对于 RabbitMQ 来说, VirtualHost 也是可以随意创建删除的.

此处咱们暂时不做这部分功能(实现起来也比较简单, 咱们的代码中会完成部分和虚拟主机相关的结构设计. 大家可以自行完成管理逻辑).

交换机类型 (Exchange Type)

对于 RabbitMQ 来说, 主要支持四种交换机类型.

- Direct
- Fanout
- Topic
- Header

其中 Header 这种方式比较复杂, 比较少见. 常用的是前三种交换机类型. 咱们此处也主要实现这三种.

- Direct: 生产者发送消息时, 直接指定被该交换机绑定的队列名.
- Fanout: 生产者发送的消息会被复制到该交换机的所有队列中.
- Topic: 绑定队列到交换机上时, 指定一个字符串为 bindingKey. 发送消息指定一个字符串为 routingKey. 当 routingKey 和 bindingKey 满足一定的匹配条件的时候, 则把消息投递到指定队列.

这三种操作就像给 qq 群发红包.

- Direct 是发一个专属红包, 只有指定的人能领.
- Fanout 是使用了魔法, 发一个 10 块钱红包, 群里的每个人都能领 10 块钱.
- Topic 是发一个画图红包, 发 10 块钱红包, 同时出个题, 得画的像的人, 才能领. 也是每个领到的人都能领 10 块钱.

持久化

Exchange, Queue, Binding, Message 都有持久化需求.

当程序重启 / 主机重启, 保证上述内容不丢失.

网络通信

生产者和消费者都是客户端程序, broker 则是作为服务器. 通过网络进行通信.

在网络通信的过程中, 客户端部分要提供对应的 api, 来实现对服务器的操作.

1. 创建 Connection

2. 关闭 Connection
3. 创建 Channel
4. 关闭 Channel
5. 创建队列 (queueDeclare)
6. 销毁队列 (queueDelete)
7. 创建交换机 (exchangeDeclare)
8. 销毁交换机 (exchangeDelete)
9. 创建绑定 (queueBind)
10. 解除绑定 (queueUnbind)
11. 发布消息 (basicPublish)
12. 订阅消息 (basicConsume)
13. 确认消息 (basicAck)

可以看到, 在 broker 的基础上, 客户端还要增加 Connection 操作和 Channel 操作.

Connection 对应一个 TCP 连接.

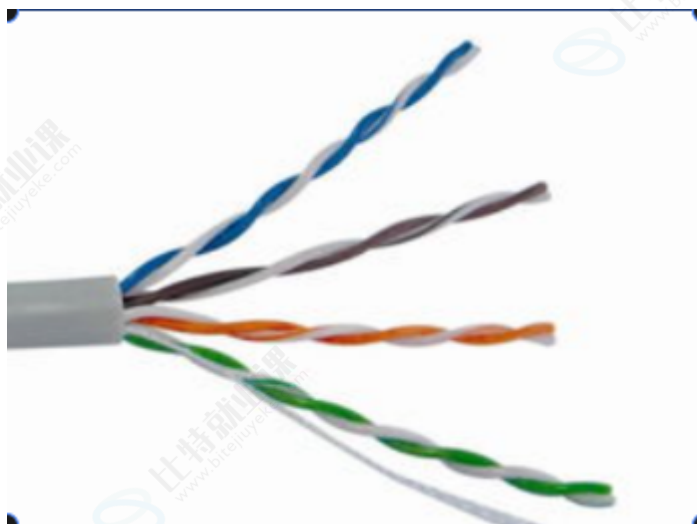
Channel 则是 Connection 中的逻辑通道.

一个 Connection 中可以包含多个 Channel.

Channel 和 Channel 之间的数据是独立的. 不会相互干扰.

这样的设定主要是为了能够更好的复用 TCP 连接, 达到长连接的效果, 避免频繁的创建关闭 TCP 连接.

Connection 可以理解成一根网线. Channel 则是网线里具体的线缆.



消息应答

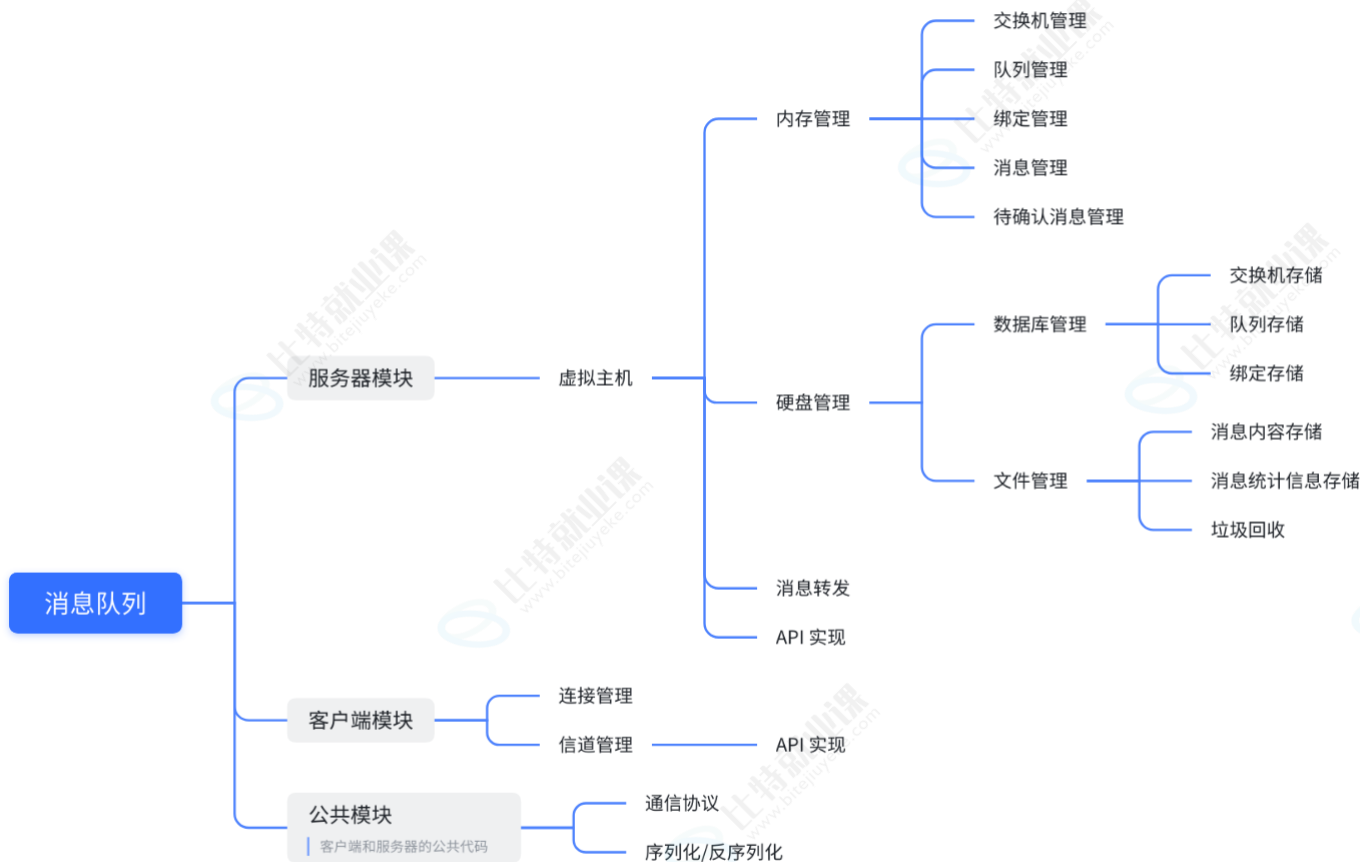
被消费的消息, 需要进行应答.

应答模式分成两种.

- 自动应答: 消费者只要消费了消息, 就算应答完毕了. Broker 直接删除这个消息.
- 手动应答: 消费者手动调用应答接口, Broker 收到应答请求之后, 才真正删除这个消息.

手动应答的目的, 是为了保证消息确实被消费者处理成功了. 在一些对于数据可靠性要求高的场景, 比较常见.

三. 模块划分



可以看到, 像 交换机, 队列, 绑定, 消息, 这几个核心概念在内存和硬盘中都是存储了的.

其中内存为主, 是用来实现消息转发的关键; 硬盘为辅, 主要是保证服务器重启之后, 之前的信息都可以正常保持.

四. 项目创建

创建 SpringBoot 项目.

使用 SpringBoot 2 系列版本, Java 8.

依赖引入 Spring Web 和 MyBatis.

五. 创建核心类

创建包 `mqserver.core`

创建 Exchange

```
1 public class Exchange {
2     private String name;
3     private ExchangeType type = ExchangeType.DIRECT;
4     private boolean durable = false;
5     private boolean autoDelete = false;
6     private Map<String, Object> arguments = new HashMap<>();
7
8     // 省略 getter setter
9 }
```

```
1 public enum ExchangeType {
2     DIRECT(0),
3     FANOUT(1),
4     TOPIC(2);
5
6     private final int type;
7
8     private ExchangeType(int type) {
9         this.type = type;
10    }
11
12    public int getType() {
13        return this.type;
14    }
15 }
```

- `name` : 交换机的名字. 相当于交换机的身份标识.
- `type` : 交换机的类型. 三种取值, DIRECT, FANOUT, TOPIC.
- `durable` : 交换机是否要持久化存储. true 为持久化, false 不持久化.
- `autoDelete` : 使用完毕后是否自动删除. 预留字段, 暂时未使用.
- `arguments` : 交换机的其他参数属性. 预留字段, 暂时未使用.

RabbitMQ 中的交换机, 支持 `autoDelete` 和 `arguments`, 咱们此处为了简单, 暂时没有实现对应功能, 只是预留了字段, 同学们可以尝试自己完成.

创建 MSGQueue

```
1 public class MSGQueue {
2     private String name;
3     private boolean durable;
4     private boolean exclusive;
5     private boolean autoDelete;
6     private Map<String, Object> arguments = new HashMap<>();
7
8     // 省略 getter setter
9 }
```

类名叫做 MSGQueue, 而不是 Queue, 是为了防止和标准库中的 Queue 混淆.

- `name`: 队列的名字. 相当于队列的身份标识.
- `durable`: 交换机是否要持久化存储. true 为持久化, false 不持久化.
- `exclusive`: 独占(排他), 队列只能被一个消费者使用.
- `autoDelete`: 使用完毕后是否自动删除. 预留字段, 暂时未使用.
- `arguments`: 交换机的其他参数属性. 预留字段, 暂时未使用.

创建 Binding

```
1 public class Binding {
2     private String exchangeName;
3     private String queueName;
4     private String bindingKey;
5
6     // 省略 getter setter
7 }
```

- `exchangeName` 交换机名字
- `queueName` 队列名字
- `bindingKey` 只在交换机类型为 `TOPIC` 时才有效. 用于和消息中的 `routingKey` 进行匹配.

创建 Message

```
1 public class Message implements Serializable {
2     private BasicProperties basicProperties = new BasicProperties();
3     private byte[] body;
4
5     // 消息在文件中对应的 offset 的范围, [offsetBeg, offsetEnd)
6     // 从这个范围取出的 byte[] 正好可以反序列化成一个 Message 对象.
7     // offsetBeg 前面的 4 个字节是消息的长度
8     private transient long offsetBeg = 0;
9     private transient long offsetEnd = 0;
10
11     // 消息在文件中是否有效. 0x0 表示无效, 0x1 表示有效
12     private byte isValid = 0x1;
13
14     // 创建新的消息, 同时给该消息分配一个新的 messageId
15     // routingKey 以参数的为准. 会覆盖掉 basicProperties 中的 routingKey
16     public static Message createMessageWithId(String routingKey,
17         BasicProperties basicProperties, byte[] body) {
18         Message message = new Message();
19         if (basicProperties != null) {
20             message.basicProperties = basicProperties;
21         }
22         message.basicProperties.setMessageId("M-" +
23             UUID.randomUUID().toString());
24         message.basicProperties.setRoutingKey(routingKey);
25         message.body = body;
26         return message;
27     }
28     // 省略 getter setter
29 }
```

```
1 public class BasicProperties implements Serializable {
2     // 消息的唯一 id. 使用 uuid 表示.
3     private String messageId;
4     private String routingKey;
5     // 1 表示消息非持久化. 2 表示消息持久化
6     private int deliveryMode = 1;
7
8     // 省略 getter setter
9 }
```

- `Message` 需要实现 `Serializable` 接口. 后续需要把 `Message` 写入文件以及进行网络传输.
- `basicProperties` 是消息的属性信息. `body` 是消息体.
- `offsetBeg` 和 `offsetEnd` 表示消息在消息文件中所在的起始位置和结束位置. 这一块具体的设计后面再详细介绍. 使用 `transient` 关键字避免属性被序列化.
- `isValid` 用来表示消息在文件中是否有效. 这一块具体的设计后面再详细介绍.
- `createMessageWithId` 相当于一个工厂方法, 用来创建一个 `Message` 实例. `messageId` 通过 `UUID` 的方式生成.

六. 数据库设计

对于 `Exchange`, `MSGQueue`, `Binding`, 我们使用数据库进行持久化保存.

此处我们使用的数据库是 `SQLite`, 是一个更轻量的数据库.

`SQLite` 只是一个动态库(当然, 官方也提供了可执行程序 `exe`), 我们在 `Java` 中直接引入 `SQLite` 依赖, 即可直接使用, 不必安装其他的软件.

配置 `sqlite`

引入 `pom.xml` 依赖

```
1 <dependency>
2   <groupId>org.xerial</groupId>
3   <artifactId>sqlite-jdbc</artifactId>
4   <version>3.41.0.1</version>
5 </dependency>
```

配置数据源 `application.yml`

```
1 spring:
2   datasource:
3     url: jdbc:sqlite:./data/meta.db
4     username:
5     password:
6     driver-class-name: org.sqlite.JDBC
7
8   mybatis:
9     mapper-locations: classpath:mapper/**Mapper.xml
```

Username 和 password 空着即可.

此处我们约定, 把数据库文件放到 `./data/meta.db` 中.

SQLite 只是把数据单纯的存储到一个文件中. 非常简单方便.

实现创建表

```
1 @Mapper
2 public interface MetaMapper {
3     void createUserTable();
4     void createExchangeTable();
5     void createQueueTable();
6     void createBindingTable();
7 }
```

本身 MyBatis 针对 MySQL / Oracle 支持执行多个 SQL 语句的, 但是针对 SQLite 是不支持的, 只能写成多个方法.

```
1 <update id="createExchangeTable">
2     create table if not exists exchange (
3         name varchar(50) primary key,
4         type int,                                -- 0 表示 direct, 1 表示 fanout, 2 表示
        topic
5         durable boolean,                        -- false 表示不持久化, true 表示持久化.
6         autoDelete boolean,                    -- false 表示不自动删除, true 表示自动删除.
7         arguments varchar(1024)                -- 创建交换机指定的参数
8     );
9 </update>
10
11 <update id="createQueueTable">
12     create table if not exists queue (
13         name varchar(50) primary key,
14         durable boolean,                        -- false 表示不持久化, true 表示持久化.
15         autoDelete boolean,                    -- false 表示不自动删除, true 表示自动删除.
16         arguments varchar(1024)                -- 创建交换机指定的参数
17     );
18 </update>
19
20 <update id="createBindingTable">
21     create table if not exists binding (
```

```
22         exchangeName varchar(50),
23         queueName varchar(50),
24         bindingKey varchar(256)
25     );
26 </update>
```

实现数据库基本操作

给 `mapper.MetaMapper` 中添加

```
1 void insertExchange(Exchange exchange);
2 void deleteExchange(String exchangeName);
3 void insertQueue(MSGQueue msgQueue);
4 void deleteQueue(String queueName);
5 void insertBinding(Binding binding);
6 void deleteBinding(Binding binding);
```

给 `MetaMapper` 中添加

```
1 <insert id="insertExchange"
   parameterType="com.example.java_message_queue.mqserver.core.Exchange">
2     insert into exchange values(#{name}, #{type}, #{durable}, #{autoDelete}, #
   {arguments});
3 </insert>
4
5 <delete id="deleteExchange" parameterType="java.lang.String">
6     delete from exchange where name = #{exchangeName};
7 </delete>
8
9 <insert id="insertQueue"
   parameterType="com.example.java_message_queue.mqserver.core.MSGQueue">
10    insert into queue values(#{name}, #{durable}, #{autoDelete}, #{arguments});
11 </insert>
12
13 <delete id="deleteQueue" parameterType="java.lang.String">
14    delete from queue where name = #{queueName};
15 </delete>
16
17 <insert id="insertBinding"
   parameterType="com.example.java_message_queue.mqserver.core.Binding">
```

```

18     insert into binding values("#{exchangeName}", "#{queueName}", "#{bindingKey}");
19 </insert>
20
21 <delete id="deleteBinding"
    parameterType="com.example.java_message_queue.mqserver.core.Binding">
22     delete from binding where exchangeName = #{exchangeName} and queueName = #
        {queueName};
23 </delete>

```

实现 DataBaseManager

mqserver.datacenter.DataBaseManager

1) 创建 DataBaseManager 类

通过这个类来封装针对数据库的操作。

```

1 public class DataBaseManager {
2     // 由于 DataBaseManager 不是一个 Bean
3     // 需要手动来获取实例
4     private MetaMapper metaMapper;
5
6     public void init() {
7         this.metaMapper =
8             JavaMessageQueueApplication.ac.getBean(MetaMapper.class);
9
10        // 构造数据库
11        if (!checkDBExists()) {
12            // 1. 读取 sql 文件中的内容, 并创建表
13            createTable();
14            // 2. 插入默认数据
15            createDefaultData();
16            System.out.println("[DataBaseManager] 数据库初始化完成!");
17        } else {
18            System.out.println("[DataBaseManager] 数据库已经存在!");
19        }
20    }
21 }

```

如果数据库文件存在, 则不必建库建表了。

针对 JavaMessageQueueApplication, 需要新增一个 ac 属性. 并初始化

```
1 @SpringBootApplication
2 public class JavaMessageQueueApplication {
3     public static ConfigurableApplicationContext ac;
4
5     public static void main(String[] args) throws IOException {
6         ac = SpringApplication.run(JavaMessageQueueApplication.class);
7     }
8 }
```

2) 实现 checkDBExists

```
1 private boolean checkDBExists() {
2     File file = new File("./meta.db");
3     if (file.exists()) {
4         return true;
5     }
6     return false;
7 }
```

3) 实现 createTable

```
1 // 创建数据表
2 private void createTable() {
3     metaMapper.createExchangeTable();
4     metaMapper.createQueueTable();
5     metaMapper.createBindingTable();
6     System.out.println("[DataBaseManager] 创建表完成!");
7 }
```

4) 实现 createDefaultData

```
1 // 创建表中的默认数据
```



```

2 private void createDefaultData() {
3     // 构造默认交换机
4     Exchange exchange = new Exchange();
5     exchange.setName("");
6     exchange.setType(ExchangeType.DIRECT);
7     exchange.setDurable(true);
8     exchange.setAutoDelete(false);
9     metaMapper.insertExchange(exchange);
10    System.out.println("[DataBaseManager] 创建初始数据完成!");
11 }

```

默认数据主要是创建一个默认的交换机. 这个默认交换机没有名字, 并且是直接交换机.

5) 封装其他数据库操作

```

1 public void insertExchange(Exchange exchange) {
2     metaMapper.insertExchange(exchange);
3 }
4
5 public void deleteExchange(String exchangeName) {
6     metaMapper.deleteExchange(exchangeName);
7 }
8
9 public List<Exchange> selectAllExchanges() {
10    return metaMapper.selectAllExchanges();
11 }
12
13 public void insertQueue(MSGQueue queue) {
14     metaMapper.insertQueue(queue);
15 }
16
17 public void deleteQueue(String queueName) {
18     metaMapper.deleteQueue(queueName);
19 }
20
21 public List<MSGQueue> selectAllQueues() {
22    return metaMapper.selectAllQueues();
23 }
24
25 public void insertBinding(Binding binding) {
26     metaMapper.insertBinding(binding);
27 }
28
29 public void deleteBinding(Binding binding) {

```

```
30     metaMapper.deleteBinding(binding);
31 }
32
33 public List<Binding> selectAllBindings() {
34     return metaMapper.selectAllBindings();
35 }
```

测试 DataBaseManager

使用 Spring 自带的单元测试, 针对上述代码进行测试验证.

在 test 目录中, 创建 DataBaseManagerTests

1) 准备工作

```
1 @SpringBootTest
2 public class DataBaseManagerTests {
3     private static DataBaseManager dataBaseManager = new DataBaseManager();
4
5     @BeforeAll
6     public static void setupAll() throws IOException {
7         // 初始情况下, 先统一清除数据库
8         dataBaseManager.deleteDB();
9     }
10
11     @BeforeEach
12     public void setUp() throws IOException {
13         // 每次运行一个用例, 都重置数据库. 防止用例之间的数据相互干扰.
14         // 需要初始化 ac 对象
15         JavaMessageQueueApplication.ac =
16             SpringApplication.run(JavaMessageQueueApplication.class);
17         // 再初始化数据库
18         dataBaseManager.init();
19     }
20
21     @AfterEach
22     public void tearDown() throws IOException {
23         // 需要关闭 ac 对象
24         JavaMessageQueueApplication.ac.close();
25         // 然后再删除数据库
26         dataBaseManager.deleteDB();
27     }
28 }
```

- `@SpringBootTest` 注解表示该类是一个测试类。
- `@BeforeAll` 在所有测试执行之前执行. 此处先删除之前的数据库, 避免干扰。
- `@BeforeEach` 每个测试用例之前执行. 一般用来做准备工作. 此处进行数据库初始化, 以及针对 Spring 服务的初始化。
- `@AfterEach` 每个测试用例之后执行. 一般用来做收尾工作. 此处需要先关闭 Spring 服务, 再删除数据库。

由于 Spring 服务启动的时候, 会和数据库建立连接(通过 MyBatis). 因此需要先关闭服务, 才能删除数据库, 否则会删除失败(Spring 服务会持有数据库文件的访问权限)。

2) 编写测试用例

- `@Test` 注解表示一个测试用例。
- `Assertions` 是断言, 用来断定执行结果。
- 每个用例执行之前, 都会自动调用到 `setUp`, 每次用例执行结束之后, 都会自动调用 `tearDown`
- 要确保每个用例的执行都是 "clean" 的, 也就是该用例不会被上个用例干扰到。

```

1 @Test
2 public void testInitTable() throws IOException {
3     List<Exchange> exchangeList = dataBaseManager.selectAllExchanges();
4     List<MSGQueue> queueList = dataBaseManager.selectAllQueues();
5     List<Binding> bindingList = dataBaseManager.selectAllBindings();
6
7     Assertions.assertEquals(1, exchangeList.size());
8     Assertions.assertEquals("", exchangeList.get(0).getName());
9     Assertions.assertEquals(ExchangeType.DIRECT,
10         exchangeList.get(0).getType());
11     Assertions.assertEquals(0, queueList.size());
12     Assertions.assertEquals(0, bindingList.size());
13 }
14 private Exchange createTestExchange(String exchangeName) {
15     Exchange exchange = new Exchange();
16     exchange.setName(exchangeName);
17     exchange.setType(ExchangeType.FANOUT);
18     exchange.setAutoDelete(true);
19     exchange.setDurable(true);
20     HashMap<String, Object> arguments = new HashMap<>();
21     arguments.put("aaa", "111");

```

```

22     arguments.put("bbb", "222");
23     exchange.setArguments(arguments);
24     return exchange;
25 }
26
27 @Test
28 public void testInsertExchange() {
29     Exchange exchange = createTestExchange("test");
30     dataBaseManager.insertExchange(exchange);
31     List<Exchange> exchangeList = dataBaseManager.selectAllExchanges();
32     Assertions.assertEquals(2, exchangeList.size());
33     Assertions.assertEquals("test", exchangeList.get(1).getName());
34     Assertions.assertEquals(ExchangeType.FANOUT,
exchangeList.get(1).getType());
35     Assertions.assertEquals(true, exchangeList.get(1).isAutoDelete());
36     Assertions.assertEquals(true, exchangeList.get(1).isDurable());
37     Assertions.assertEquals("111", exchangeList.get(1).getArgument("aaa"));
38     Assertions.assertEquals("222", exchangeList.get(1).getArgument("bbb"));
39 }
40
41 @Test
42 public void testDeleteExchange() {
43     Exchange exchange = createTestExchange("test");
44     dataBaseManager.insertExchange(exchange);
45     List<Exchange> exchangeList = dataBaseManager.selectAllExchanges();
46     Assertions.assertEquals(2, exchangeList.size());
47     Assertions.assertEquals("test", exchangeList.get(1).getName());
48
49     dataBaseManager.deleteExchange("test");
50     exchangeList = dataBaseManager.selectAllExchanges();
51     Assertions.assertEquals(1, exchangeList.size());
52     Assertions.assertEquals("", exchangeList.get(0).getName());
53 }
54
55 private MSGQueue createTestQueue(String queueName) {
56     MSGQueue queue = new MSGQueue();
57     queue.setName(queueName);
58     queue.setDurable(true);
59     queue.setAutoDelete(true);
60     queue.setExclusive(true);
61     HashMap<String, Object> hashMap = new HashMap<>();
62     hashMap.put("aaa", "111");
63     hashMap.put("bbb", "222");
64     queue.setArguments(hashMap);
65     return queue;
66 }
67

```

```
68 @Test
69 public void testInsertQueue() {
70     MSGQueue queue = createTestQueue("test");
71     dataBaseManager.insertQueue(queue);
72     List<MSGQueue> queueList = dataBaseManager.selectAllQueues();
73     Assertions.assertEquals(1, queueList.size());
74     Assertions.assertEquals("test", queueList.get(0).getName());
75     Assertions.assertEquals(true, queueList.get(0).isDurable());
76     Assertions.assertEquals(true, queueList.get(0).isAutoDelete());
77     Assertions.assertEquals(true, queueList.get(0).isExclusive());
78     Assertions.assertEquals("111", queueList.get(0).getArgument("aaa"));
79     Assertions.assertEquals("222", queueList.get(0).getArgument("bbb"));
80 }
81
82 @Test
83 public void testDeleteQueue() {
84     MSGQueue queue = createTestQueue("test");
85     dataBaseManager.insertQueue(queue);
86     List<MSGQueue> queueList = dataBaseManager.selectAllQueues();
87     Assertions.assertEquals(1, queueList.size());
88     Assertions.assertEquals("test", queueList.get(0).getName());
89
90     dataBaseManager.deleteQueue("test");
91     queueList = dataBaseManager.selectAllQueues();
92     Assertions.assertEquals(0, queueList.size());
93 }
94
95 @Test
96 public void testInsertBinding() {
97     Binding binding = new Binding();
98     binding.setQueueName("testQueue");
99     binding.setExchangeName("testExchange");
100    binding.setBindingKey("testBindingKey");
101    dataBaseManager.insertBinding(binding);
102
103    List<Binding> bindingList = dataBaseManager.selectAllBindings();
104    Assertions.assertEquals(1, bindingList.size());
105    Assertions.assertEquals("testQueue", bindingList.get(0).getQueueName());
106    Assertions.assertEquals("testExchange",
        bindingList.get(0).getExchangeName());
107    Assertions.assertEquals("testBindingKey",
        bindingList.get(0).getBindingKey());
108 }
109
110 @Test
111 public void testDeleteBinding() {
112     Binding binding = new Binding();
```

```
113 binding.setQueueName("testQueue");
114 binding.setExchangeName("testExchange");
115 binding.setBindingKey("testBindingKey");
116 dataBaseManager.insertBinding(binding);
117
118 List<Binding> bindingList = dataBaseManager.selectAllBindings();
119 Assertions.assertEquals(1, bindingList.size());
120
121 dataBaseManager.deleteBinding(binding);
122
123 bindingList = dataBaseManager.selectAllBindings();
124 Assertions.assertEquals(0, bindingList.size());
125 }
```

七. 消息存储设计

设计思路

消息需要在硬盘上存储. 但是并不直接放到数据库中, 而是直接使用文件存储.

原因如下:

1. 对于消息的操作并不需要复杂的 增删改查 .
2. 对于文件的操作效率比数据库会高很多.

主流 MQ 的实现(包括 RabbitMQ), 都是把消息存储在文件中, 而不是数据库中.

我们给每个队列分配一个目录. 目录的名字为 data + 队列名. 形如 `./data/testQueue`

该目录中包含两个固定名字的文件.

- `queue_data.txt` 消息数据文件, 用来保存消息内容.
- `queue_stat.txt` 消息统计文件, 用来保存消息统计信息.

`queue_data.txt` 文件格式:

使用二进制方式存储.

每个消息分成两个部分:

- 前四个字节, 表示 Message 对象的长度(字节数)
- 后面若干字节, 表示 Message 内容.
- 消息和消息之间首尾相连.

每个 Message 基于 Java 标准库的 `ObjectInputStream` / `ObjectOutputStream` 序列化。

Message 对象中的 `offsetBeg` 和 `offsetEnd` 正是用来描述每个消息体所在的位置。



`queue_stat.txt` 文件格式:

使用文本方式存储。

文件中只包含一行, 里面包含两列(都是整数), 使用 `\t` 分割。

第一列表示当前总的消息数目。第二列表示有效消息数目。

形如:

```
1 2000\t1500
```

创建 MessageFileManager 类

创建 `mqserver.database.MessageFileManager`

```
1 public class MessageFileManager {
2     // 表示消息的统计信息
3     static public class Stat {
4         public int totalCount;
5         public int validCount;
6     }
7
8     public void init() {
9         // 当前这里不需要做任何工作。
10    }
11
12    // 队列目录
13    private String getQueueDir(String queueName) {
14        return "./data/" + queueName;
15    }
16
17    // 队列数据文件
```

```

18 // 这个文件来存储队列的真实数据
19 private String getQueueDataPath(String queueName) {
20     return getQueueDir(queueName) + "/queue_data.txt";
21 }
22
23 // 队列统计文件
24 // 这个文件用来存储队列中的统计信息.
25 // 包含一行, 两个列使用 \t 分割, 分别是总数据, 和无效数据.
26 private String getQueueStatPath(String queueName) {
27     return getQueueDir(queueName) + "/queue_stat.txt";
28 }
29 }

```

- 内部包含一个 Stat 类, 用来表示消息统计文件的内容.
- getQueueDir, getQueueDataPath, getQueueStatPath 用来表示这几个文件所在位置.

实现统计文件读写

这是后续操作的一项准备工作.

```

1 // 从统计文件中读取结果
2 private Stat readStat(String queueName) {
3     Stat stat = new Stat();
4     try (InputStream inputStream = new
FileInputStream(getQueueStatPath(queueName))) {
5         Scanner scanner = new Scanner(inputStream);
6         stat.totalCount = scanner.nextInt();
7         stat.validCount = scanner.nextInt();
8         return stat;
9     } catch (IOException e) {
10         e.printStackTrace();
11     }
12     return null;
13 }
14
15 // 向统计文件中写入结果
16 private void writeStat(String queueName, Stat stat) {
17     try (OutputStream outputStream = new
FileOutputStream(getQueueStatPath(queueName))) {
18         PrintWriter printWriter = new PrintWriter(outputStream);
19         printWriter.write(stat.totalCount + "\t" + stat.validCount);
20         printWriter.flush();
21     } catch (IOException e) {

```



```
22         e.printStackTrace();
23     }
24 }
```

直接使用 Scanner 和 PrintWriter 进行读写即可.

实现创建队列目录

每个队列都有自己的目录和配套的文件. 通过下列方法把目录和文件先准备好.

```
1 public void createQueueFiles(String queueName) throws IOException {
2     // 1. 创建目录指定队列的目录
3     File baseDir = new File(getQueueDir(queueName));
4     if (!baseDir.exists()) {
5         boolean ok = baseDir.mkdirs();
6         if (!ok) {
7             throw new IOException("创建目录失败! baseDir=" +
baseDir.getAbsolutePath());
8         }
9     }
10    // 2. 创建队列数据文件
11    File queueDataFile = new File(getQueueDataPath(queueName));
12    if (!queueDataFile.exists()) {
13        boolean ok = queueDataFile.createNewFile();
14        if (!ok) {
15            throw new IOException("创建文件失败! queueDataFile=" +
queueDataFile.getAbsolutePath());
16        }
17    }
18    // 3. 创建队列统计文件
19    File queueStatFile = new File(getQueueStatPath(queueName));
20    if (!queueStatFile.exists()) {
21        boolean ok = queueStatFile.createNewFile();
22        if (!ok) {
23            throw new IOException("创建文件失败! queueStatFile=" +
queueStatFile.getAbsolutePath());
24        }
25    }
26    // 4. 给队列统计文件写入初始数据
27    Stat stat = new Stat();
28    stat.totalCount = 0;
29    stat.validCount = 0;
30    writeStat(queueName, stat);
31 }
```

把上述约定的文件都创建出来, 并对消息统计文件进行初始化.

初始化 `0\t0` 这样的初始值.

实现删除队列目录

如果队列需要删除, 则队列对应的目录/文件也需要删除.

```
1 public void destroyQueueFiles(String queueName) throws IOException {
2     // 1. 先删除目录中的文件
3     File queueDataFile = new File(getQueueDataPath(queueName));
4     boolean ok1 = queueDataFile.delete();
5     File queueStatFile = new File(getQueueStatPath(queueName));
6     boolean ok2 = queueStatFile.delete();
7     // 2. 再删除目录. delete 要求必须是空目录才能删除.
8     File baseDir = new File(getQueueDir(queueName));
9     boolean ok3 = baseDir.delete();
10    if (!ok1 || !ok2 || !ok3) {
11        throw new IOException("删除队列目录失败! baseDir=" +
12            baseDir.getAbsolutePath());
13    }
```

注意: File 类的 delete 方法只能删除空目录. 因此需要先把内部的文件先删除掉.

检查队列文件是否存在

判定该队列的消息文件和统计文件是否存在. 一旦出现缺失, 则不能进行后续工作.

```
1 private boolean checkFilesExists(String queueName) {
2     File queueData = new File(getQueueDataPath(queueName));
3     if (!queueData.exists()) {
4         return false;
5     }
6     File queueStat = new File(getQueueStatPath(queueName));
7     if (!queueStat.exists()) {
8         return false;
9     }
10    return true;
11 }
```

实现消息对象序列化/反序列化

Message 对象需要转成二进制写入文件. 并且也需要把文件中的二进制读出来解析成 Message 对象. 此处针对这里的逻辑进行封装.

创建 `common.BinaryTool`

```
1 public class BinaryTool {
2     public static Object fromBytes(byte[] data) throws IOException,
3     ClassNotFoundException {
4         Object object = null;
5         ByteArrayInputStream byteArrayInputStream = new
6         ByteArrayInputStream(data);
7         try (ObjectInputStream objectInputStream = new
8         ObjectInputStream(byteArrayInputStream)) {
9             object = objectInputStream.readObject();
10        }
11        return object;
12    }
13
14    public static byte[] toBytes(Object object) throws IOException {
15        ByteArrayOutputStream byteArrayOutputStream = new
16        ByteArrayOutputStream();
17        try (ObjectOutputStream objectOutputStream = new
18        ObjectOutputStream(byteArrayOutputStream)) {
19            objectOutputStream.writeObject(object);
20        }
21        return byteArrayOutputStream.toByteArray();
22    }
23 }
```

- 使用 `ByteArrayInputStream` / `ByteArrayOutputStream` 针对 `byte[]` 进行封装, 方便后续操作. (这两个流对象是纯内存的, 不需要进行 `close`).
- 使用 `ObjectInputStream` / `ObjectOutputStream` 进行序列化 / 反序列化操作. 通过内部的 `readObject` / `writeObject` 即可完成对应操作.
- 此处涉及到的序列化对象, 需要实现 `Serializable` 接口. 这一点咱们的 `Message` 对象已经实现过了.

对于 `serialVersionUID`, 此处咱们暂时不需要. 大家可以自行了解 `serialVersionUID` 的用途

实现写入消息文件

```

1 public void sendMessage(MSGQueue queue, Message message) throws MqException,
  IOException {
2     if (!checkFilesExists(queue.getName())) {
3         throw new MqException("[MessageFileManager] 队列匹配的文件不存在!");
4         queueName=" + queue.getName());
5     }
6     // 1. 先把 message 转成二进制
7     byte[] messageBinary = BinaryTool.toBytes(message);
8     // 此处的锁对象以队列为维度。不同队列之间不涉及锁冲突。
9     synchronized (queue) {
10        // 2. 先获取到文件总长度
11        File queueDataFile = new File(getQueueDataPath(queue.getName()));
12        message.setOffsetBeg(queueDataFile.length() + 4);
13        message.setOffsetEnd(queueDataFile.length() + 4 +
14        messageBinary.length);
15        // 3. 写入消息数据文件
16        try (OutputStream outputStream = new FileOutputStream(queueDataFile,
17        true)) {
18            DataOutputStream dataOutputStream = new
19            DataOutputStream(outputStream);
20            // 先写入消息长度
21            dataOutputStream.writeInt(messageBinary.length);
22            // 再写入消息本体
23            dataOutputStream.write(messageBinary);
24        }
25        // 4. 写入消息统计文件
26        Stat stat = readStat(queue.getName());
27        stat.totalCount += 1;
28        stat.validCount += 1;
29        writeStat(queue.getName(), stat);
30    }
31 }

```

- 考虑线程安全, 按照队列维度进行加锁.
- 使用 DataOutputStream 进行二进制写操作. 比原生 OutputStream 要方便.
- 需要记录 Message 对象在文件中的偏移量. 后续的删除操作依赖这个偏移量定位到消息. offsetBeg 是原有文件大小的基础上, 再 + 4. 4 个字节是存放消息大小的空间. (参考上面的图).
- 写完消息, 要同时更新统计信息.

创建 `common.MqException`, 作为自定义异常类. 后续业务上出现问题, 都统一抛出这个异常.

实践中创建多个异常类, 分别表示不同异常种类是更好的做法. 此处我们只是偷懒了.

```

1 public class MqException extends Exception {
2     public MqException(String message) {
3         super(message);
4     }
5 }

```

实现删除消息

此处的删除只是"逻辑删除",即把 Message 类中的 isValid 字段设置为 0.

这样删除速度比较快. 实际的彻底删除,则通过我们自己实现的 GC 来解决.

```

1 // 把文件上的对应消息给删除掉。(标记成无效)
2 public void deleteMessage(MSGQueue queue, Message message) throws IOException,
   ClassNotFoundException {
3     synchronized (queue) {
4         try (RandomAccessFile randomAccessFile = new
   RandomAccessFile(getQueueDataPath(queue.getName()), "rw")) {
5             // 1. 先从文件中读取出 Message 的数据
6             byte[] bufferSrc = new byte[(int) (message.getOffsetEnd() -
   message.getOffsetBeg())];
7             randomAccessFile.seek(message.getOffsetBeg());
8             randomAccessFile.read(bufferSrc);
9             // 2. 转成 Message 对象
10            Message diskMessage = (Message) BinaryTool.fromBytes(bufferSrc);
11            // 3. 设置成无效.
12            diskMessage.setIsValid((byte)0x0);
13            // 4. 重新写入文件
14            byte[] bufferDest = BinaryTool.toBytes(diskMessage);
15            randomAccessFile.seek(message.getOffsetBeg());
16            randomAccessFile.write(bufferDest);
17        }
18
19        // 更新统计文件
20        Stat stat = readStat(queue.getName());
21        if (stat.validCount > 0) {
22            stat.validCount -= 1;
23        }
24        writeStat(queue.getName(), stat);
25    }
26 }

```

- 使用 RandomAccessFile 来随机访问到文件的内容.

- 根据 Message 中的 offsetBeg 和 offsetEnd 定位到消息在文件中的位置. 通过 randomAccessFile.seek 操作文件指针偏移过去. 再读取.
- 读出的结果解析成 Message 对象, 修改 isValid 字段, 再重新写回文件. 注意写的时候要重新设定文件指针的位置. 文件指针会随着上述的读操作产生改变.
- 最后, 要记得更新统计文件, 把合法消息 - 1.

实现消息加载

把消息内容从文件加载到内存中. 这个功能在服务器重启, 和垃圾回收的时候都很关键.

```
1 // 从消息数据文件中读取所有消息
2 public LinkedList<Message> loadAllMessageFromQueue(String queueName) throws
   MqException, IOException, ClassNotFoundException {
3     // 记录当前读到的数据在文件的 offset
4     long currentOffset = 0;
5     LinkedList<Message> messages = new LinkedList<>();
6     try (InputStream inputStream = new
       FileInputStream(getQueueDataPath(queueName))) {
7         DataInputStream dataInputStream = new DataInputStream(inputStream);
8         while (true) {
9             // 读到文件末尾, 会触发 EOFException
10            int messageSize = dataInputStream.readInt();
11            byte[] buffer = new byte[messageSize];
12            int actualSize = dataInputStream.read(buffer);
13            if (messageSize != actualSize) {
14                throw new MqException("[MessageFileManager] 文件格式错误!
queueName=" + queueName);
15            }
16            Message message = (Message) BinaryTool.fromBytes(buffer);
17            if (message.getIsValid() != 0x1) {
18                // 被删除的无效数据, 直接跳过. 不要忘记更新 currentOffset
19                currentOffset += 4 + messageSize;
20                continue;
21            }
22            // 计算该 message 的 offset
23            message.setOffsetBeg(currentOffset + 4);
24            message.setOffsetEnd(currentOffset + 4 + messageSize);
25            // 每个消息, 开头 4 个字节保存的是消息的长度. 接下来 [offsetBeg,
offsetEnd) 是消息体
26            currentOffset += 4 + messageSize;
27            messages.add(message);
28        }
29    } catch (EOFException e) {
```

```

30         // 数据读取完毕, 循环正常退出!
31         System.out.println("[MessageFileManager] 恢复 Message 数据完成!");
32     }
33     return messages;
34 }

```

- 使用 `DataInputStream` 读取数据. 先读 4 个字节为消息的长度, 然后再按照这个长度来读取实际消息内容.
- 读取完毕之后, 转换成 `Message` 对象.
- 同时计算出该对象的 `offsetBeg` 和 `offsetEnd`.
- 最终把结果整理成链表, 返回出去.
- 注意, 对于 `DataInputStream` 来说, 如果读取到 EOF, 会抛出一个 `EOFException`, 而不是返回特定值. 因此需要注意上述循环的结束条件.

实现垃圾回收(GC)

上述删除操作, 只是把消息在文件上标记成了无效. 并没有腾出硬盘空间. 最终文件大小可能会越积越多. 因此需要定期的进行批量清除.

此处使用类似于复制算法. 当总消息数超过 2000, 并且有效消息数目少于 50% 的时候, 就触发 GC.

GC 的时候会把所有有效消息加载出来, 写入到一个新的消息文件中, 使用新文件, 代替旧文件即可.

```

1  // 检查是否要针对文件进行 GC 操作
2  public boolean checkGC(String queueName) {
3      Stat stat = readStat(queueName);
4      if (stat.totalCount >= 2000 && (double)stat.validCount / (double)
        stat.totalCount <= 0.5) {
5          return true;
6      }
7      return false;
8  }
9
10 private String getQueueDataNewPath(String queueName) {
11     return getQueueDir(queueName) + "/queue_data_new.txt";
12 }
13
14 // 真正执行 GC 操作
15 // 使用复制算法.
16 // 先创建一个新的文件, 名字为 "queue_data_new.txt"
17 // 然后加载出旧的文件的所有有效消息内容
18 // 把这些内容写入到新的文件中.

```



```

19 // 删除旧文件, 对新文件重命名.
20 public void gc(MSGQueue queue) throws MqException, IOException,
    ClassNotFoundException {
21     synchronized (queue) {
22         long gcBeg = System.currentTimeMillis();
23         // 1. 创建一个新的文件, 名字为 "queue_data_new.txt"
24         File queueDataNew = new File(getQueueDataNewPath(queue.getName()));
25         if (queueDataNew.exists()) {
26             throw new MqException("[MessageFileManager] gc 时发现队列新数据文件已
已经存在! queueName=" + queue.getName());
27         }
28         boolean ok = queueDataNew.createNewFile();
29         if (!ok) {
30             throw new IOException("创建文件失败! queueDataNew=" +
queueDataNew.getAbsolutePath());
31         }
32         // 2. 遍历旧文件, 读取出每个对象 (只保留有效消息)
33         List<Message> messageList = loadAllMessageFromQueue(queue.getName());
34         // 3. 把有效消息写入到新的文件中.
35         try (OutputStream outputStream = new FileOutputStream(queueDataNew)) {
36             DataOutputStream dataOutputStream = new
DataOutputStream(outputStream);
37             for (Message message : messageList) {
38                 byte[] buffer = BinaryTool.toBytes(message);
39                 dataOutputStream.writeInt(buffer.length);
40                 dataOutputStream.write(buffer);
41             }
42         }
43         // 4. 删除 queue_data.txt, 把 queue_data_new.txt 重命名为 queue_data
44         File queueDataOld = new File(getQueueDataPath(queue.getName()));
45         ok = queueDataOld.delete();
46         if (!ok) {
47             throw new IOException("删除文件失败! queueDataOld=" +
queueDataOld.getAbsolutePath());
48         }
49         ok = queueDataNew.renameTo(queueDataOld);
50         if (!ok) {
51             throw new IOException("文件重命名失败! queueDataOld=" +
queueDataOld.getAbsolutePath() +
52                 ", queueDataNew=" + queueDataNew.getAbsolutePath());
53         }
54         // 5. 更新统计文件
55         Stat stat = readStat(queue.getName());
56         stat.validCount = messageList.size();
57         stat.totalCount = messageList.size();
58         writeStat(queue.getName(), stat);
59

```



```

60         long gcEnd = System.currentTimeMillis();
61         System.out.println("[MessageFileManager] gc 执行完毕! queueName=" +
            queue.getName() + ", time=" + (gcEnd - gcBeg) + "ms");
62     }
63 }

```

如果文件很大, 消息非常多, 可能比较低效, 这种就需要把文件做拆分和合并了。

Rabbitmq 本体是这样实现的。但是咱们此处为了实现简单, 就不做这个了。

测试 MessageFileManager

创建 `MessageFileManagerTests` 编写测试用例代码。

- 创建两个队列, 用来辅助测试。
- 使用 `ReflectionTestUtils.invokeMethod` 来调用私有方法。

```

1 @SpringBootTest
2 public class MessageFileManagerTests {
3     private String queueName1 = "testQueue1";
4     private String queueName2 = "testQueue2";
5     private MessageFileManager messageFileManager = new MessageFileManager();
6
7
8     @BeforeEach
9     public void setUp() throws IOException {
10         messageFileManager.createQueueFiles(queueName1);
11         messageFileManager.createQueueFiles(queueName2);
12     }
13
14     @AfterEach
15     public void tearDown() throws IOException {
16         messageFileManager.destroyQueueFiles(queueName1);
17         messageFileManager.destroyQueueFiles(queueName2);
18     }
19 }

```

```

1 @Test
2 public void testCreateFile() {
3     File queueDataFile1 = new File("./data/" + queueName1 + "/queue_data.txt");
4     Assertions.assertEquals(true, queueDataFile1.isFile());
5     File queueStatFile1 = new File("./data/" + queueName1 + "/queue_stat.txt");

```

```
6     Assertions.assertEquals(true, queueStatFile1.isFile());
7     Assertions.assertTrue(queueStatFile1.length() > 0);
8     File queueDataFile2 = new File("./data/" + queueName2 + "/queue_data.txt");
9     Assertions.assertEquals(true, queueDataFile2.isFile());
10    File queueStatFile2 = new File("./data/" + queueName2 + "/queue_stat.txt");
11    Assertions.assertEquals(true, queueStatFile2.isFile());
12    Assertions.assertTrue(queueStatFile2.length() > 0);
13 }
14
15 @Test
16 public void testReadWriteStat() {
17     MessageFileManager.Stat stat = new MessageFileManager.Stat();
18     stat.totalCount = 100;
19     stat.validCount = 50;
20     // 通过 Spring 提供的反射工具类, 调用私有方法.
21     ReflectionTestUtils.invokeMethod(messageFileManager, "writeStat",
queueName1, stat);
22     MessageFileManager.Stat newStat =
ReflectionTestUtils.invokeMethod(messageFileManager, "readStat", queueName1);
23     Assertions.assertEquals(100, newStat.totalCount);
24     Assertions.assertEquals(50, newStat.validCount);
25 }
26
27 private MSGQueue createTestQueue(String queueName) {
28     MSGQueue queue = new MSGQueue();
29     queue.setName(queueName);
30     queue.setDurable(true);
31     queue.setAutoDelete(true);
32     queue.setExclusive(true);
33     HashMap<String, Object> hashMap = new HashMap<>();
34     hashMap.put("aaa", "111");
35     hashMap.put("bbb", "222");
36     queue.setArguments(hashMap);
37     return queue;
38 }
39
40 private Message createTestMessage(String content) {
41     Message message = new Message();
42     message.setMessageId("M-" + UUID.randomUUID().toString());
43     message.setRoutingKey("testRoutingKey");
44     message.setDeliveryMode(2);
45     message.setBody(content.getBytes());
46     return message;
47 }
48
49 @Test
```

```
50 public void testSendMessage() throws IOException, MqException,
    ClassNotFoundException {
51     Message message = createTestMessage("testMessage");
52     MSGQueue queue = createTestQueue(queueName1);
53     messageFileManager.sendMessage(queue, message);
54
55     // 检查 stat 文件
56     MessageFileManager.Stat newStat =
    ReflectionTestUtils.invokeMethod(messageFileManager, "readStat", queueName1);
57     Assertions.assertEquals(1, newStat.totalCount);
58     Assertions.assertEquals(1, newStat.validCount);
59
60     // 读文件内容
61     List<Message> messageList =
    messageFileManager.loadAllMessageFromQueue(queueName1);
62     Assertions.assertEquals(1, messageList.size());
63     Message curMessage = messageList.get(0);
64     Assertions.assertEquals(message.getMessageId(), curMessage.getMessageId());
65     Assertions.assertEquals(message.getRoutingKey(),
    curMessage.getRoutingKey());
66     Assertions.assertEquals(message.getDeliveryMode(),
    curMessage.getDeliveryMode());
67     Assertions.assertArrayEquals(message.getBody(), curMessage.getBody());
68 }
69
70 @Test
71 public void testLoadAllMessageFromQueue() throws IOException, MqException,
    ClassNotFoundException {
72     MSGQueue queue = createTestQueue(queueName1);
73     List<Message> expectedMessages = new ArrayList<>();
74     for (int i = 0; i < 100; i++) {
75         Message message = createTestMessage("testMessage");
76         messageFileManager.sendMessage(queue, message);
77         expectedMessages.add(message);
78     }
79     List<Message> actualMessages =
    messageFileManager.loadAllMessageFromQueue(queueName1);
80     Assertions.assertEquals(100, actualMessages.size());
81     for (int i = 0; i < 100; i++) {
82         Message expectedMessage = actualMessages.get(i);
83         Message actualMessage = actualMessages.get(i);
84         System.out.println "[" + i + " ] " + actualMessage;
85         Assertions.assertEquals(expectedMessage.getMessageId(),
    actualMessage.getMessageId());
86         Assertions.assertEquals(expectedMessage.getRoutingKey(),
    actualMessage.getRoutingKey());
```

```
87     Assertions.assertEquals(expectedMessage.getDeliveryMode(),
88         actualMessage.getDeliveryMode());
89     Assertions.assertEquals(0x1, actualMessage.getIsValid());
90 }
91 }
92
93 @Test
94 public void testDeleteMessage() throws IOException, MqException,
    ClassNotFoundException {
95     MSGQueue queue = createTestQueue(queueName1);
96     List<Message> expectedMessages = new ArrayList<>();
97     for (int i = 0; i < 10; i++) {
98         Message message = createTestMessage("testMessage");
99         messageFileManager.sendMessage(queue, message);
100         expectedMessages.add(message);
101     }
102     System.out.println("expected:" + expectedMessages);
103
104     messageFileManager.deleteMessage(queue, expectedMessages.get(0));
105     messageFileManager.deleteMessage(queue, expectedMessages.get(1));
106     messageFileManager.deleteMessage(queue, expectedMessages.get(2));
107
108     // 读出来, 这个方法只能加载有效数据.
109     List<Message> actualMessages =
110         messageFileManager.loadAllMessageFromQueue(queueName1);
111     System.out.println("actual: " + actualMessages);
112     Assertions.assertEquals(7, actualMessages.size());
113     for (int i = 0; i < actualMessages.size(); i++) {
114         Assertions.assertEquals(expectedMessages.get(i + 3).getMessageId(),
115             actualMessages.get(i).getMessageId());
116     }
117 }
118
119 @Test
120 public void testGc() throws IOException, MqException, ClassNotFoundException {
121     MSGQueue queue = createTestQueue(queueName1);
122     List<Message> expectedMessages = new ArrayList<>();
123     // 创建 100 个元素
124     for (int i = 0; i < 100; i++) {
125         Message message = createTestMessage("testMessage");
126         messageFileManager.sendMessage(queue, message);
127         expectedMessages.add(message);
128     }
129     // 删除 偶数 下标的元素
130     for (int i = 0; i < 100; i += 2) {
```

```

129     messageFileManager.deleteMessage(queue, expectedMessages.get(i));
130 }
131 // 获取旧文件大小
132 File oldFile = new File("./data/" + queueName1 + "/queue_data.txt");
133 long oldLength = oldFile.length();
134
135 // 调用 gc
136 messageFileManager.gc(queue);
137 // 重新读文件
138 List<Message> actualMessages =
    messageFileManager.loadAllMessageFromQueue(queueName1);
139 Assertions.assertEquals(50, actualMessages.size());
140 for (int i = 0; i < 50; i++) {
141     // 注意这里的下标换算
142     Assertions.assertEquals(expectedMessages.get(2 * i +
143 1).getMessageId(), actualMessages.get(i).getMessageId());
144 }
145 // 获取新文件大小
146 File newFile = new File("./data/" + queueName1 + "/queue_data.txt");
147 long newLength = newFile.length();
148 System.out.println("oldLength=" + oldLength);
149 System.out.println("newLength=" + newLength);
150 Assertions.assertTrue(oldLength > newLength);
151 }

```

八. 整合数据库和文件

上述代码中, 使用数据库存储了 Exchange, Queue, Binding, 使用文本文件存储了 Message.

接下来我们把两个部分整合起来, 统一进行管理.

创建 DiskDataCenter

使用 DiskDataCenter 来综合管理数据库和文本文件的内容.

DiskDataCenter 会持有 DataBaseManager 和 MessageFileManager 对象.

```

1 // 管理硬盘上的数据.
2 // 分成两个部分:
3 // 1. 数据库管理元信息
4 // 2. 文件管理消息内容
5 public class DiskDataCenter {
6     private String virtualHostName;

```

```

7
8    // 管理数据库中的元数据
9    private DataBaseManager dataBaseManager = new DataBaseManager();
10   // 管理文件中的消息数据
11   private MessageFileManager messageFileManager = new MessageFileManager();
12
13   public void init(String virtualHostName) {
14       this.virtualHostName = virtualHostName;
15
16       initDir();
17       dataBaseManager.init();
18       messageFileManager.init();
19   }
20 }

```

实现 initDir

```

1  // 初始化目录结构
2  // virtualHostName 为 default-VirtualHost
3  // 则存放数据的目录名为: ./data/default-VirtualHost/
4  private void initDir() {
5      File baseDir = new File("./data/" + virtualHostName);
6      if (!baseDir.exists()) {
7          boolean ok = baseDir.mkdirs();
8          if (ok) {
9              System.out.println("[DiskDataCenter] 初始化数据目录完成!");
10         } else {
11             System.out.println("[DiskDataCenter] 初始化数据目录失败!");
12         }
13     } else {
14         System.out.println("[DiskDataCenter] 数据目录已经存在!");
15     }
16 }

```

封装 Exchange 方法

```

1  public void insertExchange(Exchange exchange) {
2      dataBaseManager.insertExchange(exchange);
3  }
4
5  public void deleteExchange(String exchangeName) {

```

```

6     dataBaseManager.deleteExchange(exchangeName);
7 }
8
9 public List<Exchange> selectAllExchanges() {
10     return dataBaseManager.selectAllExchanges();
11 }

```

封装 Queue 方法

```

1 public void insertQueue(MSGQueue queue) throws IOException {
2     dataBaseManager.insertQueue(queue);
3     messageFileManager.createQueueFiles(queue.getName());
4 }
5
6 public void deleteQueue(String queueName) throws IOException {
7     dataBaseManager.deleteQueue(queueName);
8     messageFileManager.destroyQueueFiles(queueName);
9 }
10
11 public List<MSGQueue> selectAllQueues() {
12     return dataBaseManager.selectAllQueues();
13 }

```

- 创建/删除队列的时候同时创建/删除队列目录。

封装 Binding 方法

```

1 public void insertBinding(Binding binding) {
2     dataBaseManager.insertBinding(binding);
3 }
4
5 public void deleteBinding(Binding binding) {
6     dataBaseManager.deleteBinding(binding);
7 }
8
9 public List<Binding> selectAllBindings() {
10     return dataBaseManager.selectAllBindings();
11 }

```

封装 Message 方法

```
1 public void sendMessage(MSGQueue queue, Message message) throws MqException,
   IOException {
2     messageFileManager.sendMessage(queue, message);
3 }
4
5 public void deleteMessage(MSGQueue queue, Message message) throws MqException,
   IOException, ClassNotFoundException {
6     messageFileManager.deleteMessage(queue, message);
7
8     // 判定是否要 GC
9     if (messageFileManager.checkGC(queue.getName())) {
10         messageFileManager.gc(queue);
11     }
12 }
13
14 public LinkedList<Message> loadAllMessageFromQueue(String queueName) throws
   MqException, IOException, ClassNotFoundException {
15     return messageFileManager.loadAllMessageFromQueue(queueName);
16 }
```

- 在 deleteMessage 的时候判定是否进行 GC.

小结

通过上述封装, 把数据库和硬盘文件两部分合并成一个整体. 上层代码在调用的时候则不再关心该数据是存储在哪个部分的.

这个类的整体实现并不复杂, 关键逻辑在之前都已经准备好了.

该类我们就不单独进行单元测试了. 同学们可以自行完成.

九. 内存数据结构设计

硬盘上存储数据, 只是为了实现 "持久化" 这样的效果. 但是实际的消息存储/转发, 还是主要靠内存的结构.

对于 MQ 来说, 内存部分是更关键的, 内存速度更快, 可以达成更高的并发.

创建 MemoryDataCenter

创建 `mqserver.datacenter.MemoryDataCenter`

```
1 // 管理所有的内存数据.
2 public class MemoryDataCenter {
3     // key 是 exchangeName
4     private ConcurrentHashMap<String, Exchange> exchangeMap = new
    ConcurrentHashMap<>();
5     // key 是 queueName
6     private ConcurrentHashMap<String, MSGQueue> queueMap = new
    ConcurrentHashMap<>();
7     // 第一个 key 是 exchangeName, 第二个 key 是 queueName
8     private ConcurrentHashMap<String, HashMap<String, Binding>> bindingsMap =
    new ConcurrentHashMap<>();
9     // 保存所有消息, key 是 messageId
10    private ConcurrentHashMap<String, Message> messageMap = new
    ConcurrentHashMap<>();
11    // key 是 queueName
12    private ConcurrentHashMap<String, LinkedList<Message>> queueMessageMap =
    new ConcurrentHashMap<>();
13    // 用来存放待确认的消息
14    // key1 是 queueName, key2 是 messageId.
15    // 这个结构不需要有对应的硬盘数据. 换句话说, 如果某个消息消费了, 但是没有 ack, 这个
    时候 broker 宕机了, 那么重启 broker 之后
16    // 就把刚才的消息当做从来没消费过.
17    private ConcurrentHashMap<String, HashMap<String, Message>>
    queueMessageWaitAck = new ConcurrentHashMap<>();
18
19    public void init() {
20    }
21 }
```

- 使用四个哈希表, 管理 Exchange, Queue, Binding, Message.
- 使用一个哈希表 + 链表管理 队列 -> 消息 之间的关系.
- 使用一个哈希表 + 哈希表管理所有的未被确认的消息.

为了保证消息被正确消费了, 会使用两种方式进行确认. 自动 ACK 和 手动 ACK.

其中自动 ACK 是指当消息被消费之后, 就会立即被销毁释放.

其中手动 ACK 是指当消息被消费之后, 由消费者主动调用一个 `basicAck` 方法, 进行主动确认. 服务器收到这个确认之后, 才能真正销毁消息.

此处的 "未确认消息" 就是指在手动 ACK 模式下, 该消息还没有被调用 `basicAck`. 此时消息不能删除, 但是要和其他未消费的消息区分开. 于是另搞了个结构.

当后续 basicAck 到了, 就可以删除消息了.

封装 Exchange 方法

```
1 public void insertExchange(Exchange exchange) {
2     exchangeMap.put(exchange.getName(), exchange);
3 }
4
5 public Exchange getExchange(String exchangeName) {
6     return exchangeMap.get(exchangeName);
7 }
8
9 public void deleteExchange(String exchangeName) {
10     exchangeMap.remove(exchangeName);
11 }
```

封装 Queue 方法

```
1 public void insertQueue(MSGQueue queue) {
2     queueMap.put(queue.getName(), queue);
3 }
4
5 public MSGQueue getQueue(String queueName) {
6     return queueMap.get(queueName);
7 }
8
9 public void deleteQueue(String queueName) {
10     queueMap.remove(queueName);
11 }
```

封装 Binding 方法

```
1 public void insertBinding(Binding binding) throws MqException {
2     HashMap<String, Binding> bindingMap =
3         bindingsMap.computeIfAbsent(binding.getExchangeName(), k -> new HashMap<>());
4     synchronized (bindingMap) {
```

```

4      // 不存在就创建一份
5      if (bindingMap.get(binding.getQueueName()) != null) {
6          throw new MqException("[MemoryDataCenter] 绑定已经存在!
exchangeName=" + binding.getExchangeName()
7              + ", queueName=" + binding.getQueueName());
8      }
9      bindingMap.put(binding.getQueueName(), binding);
10 }
11 }
12
13 public Binding getBinding(String queueName, String exchangeName) {
14     HashMap<String, Binding> bindingMap = bindingsMap.get(exchangeName);
15     if (bindingMap == null) {
16         return null;
17     }
18     synchronized (bindingMap) {
19         return bindingMap.get(queueName);
20     }
21 }
22
23 public void deleteBinding(Binding binding) throws MqException {
24     HashMap<String, Binding> bindingMap =
bindingsMap.get(binding.getExchangeName());
25     if (bindingMap == null) {
26         throw new MqException("[MemoryDataCenter] 绑定不存在! exchangeName=" +
binding.getExchangeName()
27             + ", queueName=" + binding.getQueueName());
28     }
29     synchronized (bindingMap) {
30         Binding toDelete = bindingMap.get(binding.getQueueName());
31         if (toDelete == null) {
32             throw new MqException("[MemoryDataCenter] 绑定不存在! exchangeName="
+ binding.getExchangeName()
33                 + ", queueName=" + binding.getQueueName());
34         }
35         bindingMap.remove(binding.getQueueName());
36     }
37 }
38
39 public Map<String, Binding> getBindingsByExchange(String exchangeName) {
40     return bindingsMap.get(exchangeName);
41 }

```

```

1 // 查询指定的消息
2 public Message getMessage(String messageId) {
3     return messageMap.get(messageId);
4 }
5
6 // 向消息中心中添加消息
7 public void addMessage(Message message) {
8     messageMap.put(message.getMessageId(), message);
9     System.out.println("[MemoryCenter] 新消息被添加! messageId=" +
    message.getMessageId());
10 }
11
12 // 从消息中心删除消息
13 public void removeMessage(String messageId) {
14     messageMap.remove(messageId);
15     System.out.println("[MemoryCenter] 消息被彻底删除! messageId=" + messageId);
16 }
17
18 // 发送消息到指定队列中
19 public void sendMessage(MSGQueue queue, Message message) {
20     List<Message> messageList =
    queueMessageMap.computeIfAbsent(queue.getName(), k -> new LinkedList<>());
21     synchronized (messageList) {
22         messageList.add(message);
23     }
24     // 如果消息已经存在, 重复调用也没啥大不了的.
25     addMessage(message);
26     System.out.println("[MemoryCenter] 消息被投递到队列中! messageId=" +
    message.getMessageId() + ", queueName=" + queue.getName());
27 }
28
29 // 从指定队列中取消息.
30 public Message pollMessage(String queueName) throws MqException {
31     List<Message> messageList = queueMessageMap.get(queueName);
32     if (messageList == null) {
33         throw new MqException("[MemoryDataCenter] 队列不存在! queueName=" +
    queueName);
34     }
35     synchronized (messageList) {
36         if (messageList.size() == 0) {
37             return null;
38         }
39         // 出队列头元素
40         Message currentMessage = messageList.remove(0);
41         System.out.println("[MemoryCenter] 消息从队列中取出! messageId=" +
    currentMessage.getMessageId() + ", queueName=" + queueName);

```

```

42         return currentMessage;
43     }
44 }
45
46 public int getMessageCount(String queueName) throws MqException {
47     List<Message> messageList = queueMessageMap.get(queueName);
48     if (messageList == null) {
49         // 如果队列不存在，则直接返回长度 0，说明该 queueName 下还没有消息。
50         return 0;
51     }
52     synchronized (messageList) {
53         return messageList.size();
54     }
55 }

```

针对未确认的消息的处理

```

1 // 未被确认的消息，先临时存放一下
2 public void addMessageWaitAck(String queueName, Message message) {
3     HashMap<String, Message> messageHashMap =
4         queueMessageWaitAck.computeIfAbsent(queueName, k -> new HashMap<>());
5     synchronized (messageHashMap) {
6         messageHashMap.put(message.getMessageId(), message);
7     }
8     System.out.println("[MemoryCenter] 消息进入待确认队列! messageId=" +
9         message.getMessageId() + ", queueName=" + queueName);
10 }
11 // 消息被确认之后，就可以真正删除了。
12 public void removeMessageWaitAck(String queueName, String messageId) {
13     HashMap<String, Message> messageHashMap =
14         queueMessageWaitAck.get(queueName);
15     if (messageHashMap == null) {
16         return;
17     }
18     synchronized (messageHashMap) {
19         messageHashMap.remove(messageId);
20     }
21     System.out.println("[MemoryCenter] 消息从待确认队列删除! messageId=" +
22         messageId + ", queueName=" + queueName);
23 }
24 public Message getMessageWaitAck(String queueName, String messageId) {

```

```

23     HashMap<String, Message> messageHashMap =
        queueMessageWaitAck.get(queueName);
24     if (messageHashMap == null) {
25         return null;
26     }
27     synchronized (messageHashMap) {
28         return messageHashMap.get(messageId);
29     }
30 }

```

实现重启后恢复内存

```

1 // 从硬盘上恢复数据
2 public void recovery(DiskDataCenter diskDataCenter) throws MqException,
    IOException, ClassNotFoundException {
3     // 1. 恢复交换机数据
4     List<Exchange> exchanges = diskDataCenter.selectAllExchanges();
5     for (Exchange exchange : exchanges) {
6         exchangeMap.put(exchange.getName(), exchange);
7     }
8     // 2. 恢复队列数据
9     List<MSGQueue> queues = diskDataCenter.selectAllQueues();
10    for (MSGQueue queue : queues) {
11        queueMap.put(queue.getName(), queue);
12    }
13    // 3. 恢复绑定数据
14    List<Binding> bindings = diskDataCenter.selectAllBindings();
15    for (Binding binding : bindings) {
16        HashMap<String, Binding> bindingMap =
            bindingsMap.computeIfAbsent(binding.getExchangeName(), k -> new HashMap<>());
17        bindingMap.put(binding.getQueueName(), binding);
18    }
19    // 4. 恢复消息数据
20    // 只需要恢复 queueMessageMap 和 messageMap
21    // queueMessageWaitAck 则不必恢复。未被确认的消息只是在内存存储。如果这个时候
    broker 宕机了，则消息视为没有被消费过。
22    for (MSGQueue queue : queues) {
23        LinkedList<Message> messages =
            diskDataCenter.loadAllMessageFromQueue(queue.getName());
24        queueMessageMap.put(queue.getName(), messages);
25        for (Message message : messages) {
26            messageMap.put(message.getMessageId(), message);
27        }

```

```
28     }
29 }
```

测试 MemoryDataCenter

创建 `MemoryDataCenterTests`

```
1 @SpringBootTest
2 public class MemoryDataCenterTests {
3     private MemoryDataCenter memoryDataCenter = null;
4
5     @BeforeEach
6     public void setUp() {
7         memoryDataCenter = new MemoryDataCenter();
8         memoryDataCenter.init();
9     }
10
11     @AfterEach
12     public void tearDown() {
13         memoryDataCenter = null;
14     }
```

```
1 private Exchange createTestExchange(String exchangeName) {
2     Exchange exchange = new Exchange();
3     exchange.setName(exchangeName);
4     exchange.setType(ExchangeType.FANOUT);
5     exchange.setAutoDelete(true);
6     exchange.setDurable(true);
7     HashMap<String, Object> arguments = new HashMap<>();
8     arguments.put("aaa", "111");
9     arguments.put("bbb", "222");
10    exchange.setArguments(arguments);
11    return exchange;
12 }
13
14 private MSGQueue createTestQueue(String queueName) {
15     MSGQueue queue = new MSGQueue();
16     queue.setName(queueName);
17     queue.setDurable(true);
18     queue.setAutoDelete(true);
19     queue.setExclusive(true);
```

```

20     HashMap<String, Object> hashMap = new HashMap<>();
21     hashMap.put("aaa", "111");
22     hashMap.put("bbb", "222");
23     queue.setArguments(hashMap);
24     return queue;
25 }
26
27 @Test
28 public void testExchange() {
29     Exchange expectedExchange = createTestExchange("testExchange");
30     memoryDataCenter.insertExchange(expectedExchange);
31
32     Exchange actualExchange = memoryDataCenter.getExchange("testExchange");
33     Assertions.assertEquals(expectedExchange, actualExchange);
34
35     memoryDataCenter.deleteExchange("testExchange");
36     actualExchange = memoryDataCenter.getExchange("testExchange");
37     Assertions.assertNull(actualExchange);
38 }
39
40 @Test
41 public void testQueue() {
42     MSGQueue expectedQueue = createTestQueue("testQueue");
43     memoryDataCenter.insertQueue(expectedQueue);
44
45     MSGQueue actualQueue = memoryDataCenter.getQueue("testQueue");
46     Assertions.assertEquals(expectedQueue, actualQueue);
47
48     memoryDataCenter.deleteQueue("testQueue");
49     actualQueue = memoryDataCenter.getQueue("testQueue");
50     Assertions.assertNull(actualQueue);
51 }
52
53 @Test
54 public void testBinding() throws MqException {
55     Binding expectedBinding = new Binding();
56     expectedBinding.setQueueName("testQueue");
57     expectedBinding.setExchangeName("testExchange");
58     expectedBinding.setBindingKey("testBindingKey");
59     memoryDataCenter.insertBinding(expectedBinding);
60
61     Binding actualBinding = memoryDataCenter.getBinding("testQueue",
62 "testExchange");
63     Assertions.assertEquals(expectedBinding, actualBinding);
64
65     Map<String, Binding> bindingMap =
66     memoryDataCenter.getBindingsByExchange("testExchange");

```



```

65     actualBinding = bindingMap.get("testQueue");
66     Assertions.assertEquals(expectedBinding, actualBinding);
67
68     memoryDataCenter.deleteBinding(expectedBinding);
69     actualBinding = memoryDataCenter.getBinding("testQueue", "testExchange");
70     Assertions.assertNull(actualBinding);
71 }
72
73 private Message createTestMessage(String content) {
74     Message message = new Message();
75     message.setMessageId("M-" + UUID.randomUUID().toString());
76     message.setRoutingKey("testRoutingKey");
77     message.setDeliveryMode(2);
78     message.setBody(content.getBytes());
79     return message;
80 }
81
82 @Test
83 public void testMessage() {
84     Message expectedMessage = createTestMessage("testMessage");
85     memoryDataCenter.addMessage(expectedMessage);
86
87     Message actualMessage =
88         memoryDataCenter.getMessage(expectedMessage.getMessageId());
89     Assertions.assertEquals(expectedMessage, actualMessage);
90
91     memoryDataCenter.removeMessage(expectedMessage.getMessageId());
92     actualMessage =
93         memoryDataCenter.getMessage(expectedMessage.getMessageId());
94     Assertions.assertNull(actualMessage);
95 }
96
97 @Test
98 public void testSendMessage() throws MqException {
99     MSGQueue queue = createTestQueue("testQueue");
100     List<Message> expectedMessages = new ArrayList<>();
101     for (int i = 0; i < 10; i++) {
102         Message message = createTestMessage("testMessage");
103         memoryDataCenter.sendMessage(queue, message);
104         expectedMessages.add(message);
105     }
106
107     List<Message> actualMessages = new ArrayList<>();
108     while (true) {
109         Message message = memoryDataCenter.pollMessage("testQueue");
110         if (message == null) {
111             break;
112         }
113         actualMessages.add(message);
114     }
115     Assertions.assertEquals(expectedMessages, actualMessages);
116 }

```

```
110     }
111     actualMessages.add(message);
112 }
113
114 Assertions.assertEquals(expectedMessages.size(), actualMessages.size());
115 for (int i = 0; i < expectedMessages.size(); i++) {
116     Assertions.assertEquals(expectedMessages.get(i),
117         actualMessages.get(i));
118 }
119
120 @Test
121 public void testMessageWaitAck() {
122     Message expectedMessage = createTestMessage("testMessage");
123     memoryDataCenter.addMessageWaitAck("testQueue", expectedMessage);
124     Message actualMessage = memoryDataCenter.getMessageWaitAck("testQueue",
125         expectedMessage.getMessageId());
126     Assertions.assertEquals(expectedMessage, actualMessage);
127
128     memoryDataCenter.removeMessageWaitAck("testQueue",
129         expectedMessage.getMessageId());
130     actualMessage = memoryDataCenter.getMessageWaitAck("testQueue",
131         expectedMessage.getMessageId());
132     Assertions.assertNull(actualMessage);
133 }
134
135 @Test
136 public void testRecovery() throws IOException, MqException,
137     ClassNotFoundException {
138     JavaMessageQueueApplication.ac =
139         SpringApplication.run(JavaMessageQueueApplication.class);
140
141     // 构造初始数据
142     DiskDataCenter diskDataCenter = new DiskDataCenter();
143     diskDataCenter.init("");
144
145     Exchange expectedExchange = createTestExchange("testExchange");
146     diskDataCenter.insertExchange(expectedExchange);
147
148     MSGQueue expectedQueue = createTestQueue("testQueue");
149     diskDataCenter.insertQueue(expectedQueue);
150
151     Binding expectedBinding = new Binding();
152     expectedBinding.setExchangeName("testExchange");
153     expectedBinding.setQueueName("testQueue");
154     expectedBinding.setBindingKey("testBindingKey");
155     diskDataCenter.insertBinding(expectedBinding);
```

```

151
152     Message expectedMessage = createTestMessage("testMessage");
153     diskDataCenter.sendMessage(expectedQueue, expectedMessage);
154
155     // 恢复数据
156     memoryDataCenter.recovery(diskDataCenter);
157
158     // 对比结果
159     Exchange actualExchange = memoryDataCenter.getExchange("testExchange");
160     Assertions.assertEquals(expectedExchange.getType(),
161         actualExchange.getType());
161     Assertions.assertEquals(expectedExchange.isDurable(),
162         actualExchange.isDurable());
162     Assertions.assertEquals(expectedExchange.isAutoDelete(),
163         actualExchange.isAutoDelete());
163     Assertions.assertEquals(expectedExchange.getArguments(),
164         actualExchange.getArguments());
164
165     MSGQueue actualQueue = memoryDataCenter.getQueue("testQueue");
166     Assertions.assertEquals(expectedQueue.isDurable(),
167         actualQueue.isDurable());
167     Assertions.assertEquals(expectedQueue.isAutoDelete(),
168         actualQueue.isAutoDelete());
168     Assertions.assertEquals(expectedQueue.isExclusive(),
169         actualQueue.isExclusive());
169     Assertions.assertEquals(expectedQueue.getArguments(),
170         actualQueue.getArguments());
170
171     Binding actualBinding = memoryDataCenter.getBinding("testQueue",
172         "testExchange");
172     Assertions.assertEquals(expectedBinding.getBindingKey(),
173         actualBinding.getBindingKey());
173
174     // 清理
175     JavaMessageQueueApplication.ac.close();
176     File dbFile = new File("meta.db");
177     dbFile.delete();
178     File dataFile = new File("./data");
179     FileUtils.deleteDirectory(dataFile);
180 }

```

十. 虚拟主机设计

至此, 内存和硬盘的数据都已经组织完成. 接下来使用 "虚拟主机" 这个概念, 把这两部分的数据也串起来.

并且实现一些 MQ 的关键 API.

注意: 在 RabbitMQ 中, 虚拟主机是可以随意创建/删除的. 咱们此处为了实现简单, 并没有实现虚拟主机的管理. 因此我们默认就只有一个虚拟主机的存在. 但是在数据结构的设计上我们预留了对于多虚拟主机的管理.

保证不同虚拟主机中的 Exchange, Queue, Binding, Message 都是相互隔离的.

创建 VirtualHost

创建 `mqserver.VirtualHost`

```
1 public class VirtualHost {
2     private String virtualHostName;
3     private DiskDataCenter diskDataCenter = new DiskDataCenter();
4     private MemoryDataCenter memoryDataCenter = new MemoryDataCenter();
5     private Router router = new Router();
6     private ConsumerManager consumerManager = new ConsumerManager(this);
7 }
```

其中 Router 用来定义转发规则, ConsumerManager 用来实现消息消费. 这两个内容后续再介绍

实现构造方法和 getter

构造方法中会针对 DiskDataCenter 和 MemoryDataCenter 进行初始化.

同时会把硬盘的数据恢复到内存中.

```
1 public VirtualHost(String virtualHostName) {
2     this.virtualHostName = virtualHostName;
3
4     // 先初始化硬盘数据
5     diskDataCenter.init(virtualHostName);
6     // 后初始化内存数据
7     memoryDataCenter.init();
8     try {
9         // 进行恢复操作
10        memoryDataCenter.recovery(diskDataCenter);
11    }
```

```

11     } catch (Exception e) {
12         e.printStackTrace();
13         System.out.println("[VirtualHost] 恢复内存数据失败!");
14     }
15 }
16
17 public String getVirtualHostName() {
18     return virtualHostName;
19 }
20
21 public DiskDataCenter getDiskDataCenter() {
22     return diskDataCenter;
23 }
24
25 public MemoryDataCenter getMemoryDataCenter() {
26     return memoryDataCenter;
27 }

```

创建交换机

- 此处的 autoDelete, arguments 其实并没有使用. 只是先预留出来. (RabbitMQ 是支持的).
- 约定, 交换机/队列的名字, 都加上 VirtualHostName 作为前缀. 这样不同 VirtualHost 中就可以存在同名的交换机或者队列了.
- exchangeDeclare 的语义是, 不存在就创建, 存在则直接返回. 因此不叫做 "exchangeCreate".
- 先写硬盘, 后写内存. 因为写硬盘失败概率更大. 如果硬盘写失败了, 也就不必写内存了.

```

1 // 创建交换机
2 // 先写硬盘, 后写内存. 写硬盘失败概率更大, 如果异常了, 也就不写内存了.
3 public boolean exchangeDeclare(String exchangeName, ExchangeType exchangeType,
4     boolean durable, boolean autoDelete,
5     Map<String, Object> arguments) {
6     // 真实的 exchangeName 需要拼接上 virtualHostName
7     exchangeName = virtualHostName + exchangeName;
8     try {
9         // 1. 判定该交换机是否存在
10        Exchange existsExchange = memoryDataCenter.getExchange(exchangeName);
11        if (existsExchange != null) {
12            System.out.println("[VirtualHost] 交换机已经存在! exchangeName=" +
13                exchangeName);
14            return true;
15        }
16        // 2. 构造 Exchange 对象

```

```

15     Exchange exchange = new Exchange();
16     exchange.setName(exchangeName);
17     exchange.setType(exchangeType);
18     exchange.setDurable(durable);
19     exchange.setAutoDelete(autoDelete);
20     exchange.setArguments(arguments);
21     // 3. 把数据写入硬盘
22     if (durable) {
23         diskDataCenter.insertExchange(exchange);
24     }
25     // 4. 把数据写入内存
26     memoryDataCenter.insertExchange(exchange);
27     System.out.println("[VirtualHost] 交换机创建完成! exchangeName=" +
exchangeName);
28     return true;
29 } catch (Exception e) {
30     System.out.println("[VirtualHost] 交换机创建失败! exchangeName=" +
exchangeName);
31     e.printStackTrace();
32     return false;
33 }
34 }

```

删除交换机

```

1 // 删除交换机
2 // 先写硬盘, 后写内存. 写硬盘失败概率更大, 如果异常了, 也就不写内存了.
3 public boolean exchangeDelete(String exchangeName) {
4     // 真实的 exchangeName 需要拼接上 virtualHostName
5     exchangeName = virtualHostName + exchangeName;
6     try {
7         // 1. 先找到对应的交换机.
8         Exchange toDelete = memoryDataCenter.getExchange(exchangeName);
9         if (toDelete == null) {
10             throw new MqException("[VirtualHost] 交换机不存在, 无法删除!");
11         }
12         // 2. 删除硬盘上的交换机数据
13         if (toDelete.isDurable()) {
14             diskDataCenter.deleteExchange(exchangeName);
15         }
16         // 3. 删除内存中的交换机数据
17         memoryDataCenter.deleteExchange(exchangeName);

```

```

18         System.out.println("[VirtualHost] 交换机删除成功! exchangeName=" +
exchangeName);
19         return true;
20     } catch (Exception e) {
21         System.out.println("[VirtualHost] 交换机删除失败! exchangeName=" +
exchangeName);
22         e.printStackTrace();
23         return false;
24     }
25 }

```

创建队列

```

1 // 创建队列
2 public boolean queueDeclare(String queueName, boolean durable, boolean
exclusive, boolean autoDelete,
3                             Map<String, Object> arguments) {
4     // 真实的 queueName 需要拼接上 virtualHostName
5     queueName = virtualHostName + queueName;
6     try {
7         // 1. 判定队列是否存在
8         MSGQueue existsQueue = memoryDataCenter.getQueue(queueName);
9         if (existsQueue != null) {
10             System.out.println("[VirtualHost] 队列已经存在! queueName=" +
queueName);
11             return true;
12         }
13         // 2. 创建队列对象
14         MSGQueue queue = new MSGQueue();
15         queue.setName(queueName);
16         queue.setDurable(durable);
17         queue.setAutoDelete(autoDelete);
18         queue.setArguments(arguments);
19         // 3. 写硬盘
20         if (durable) {
21             diskDataCenter.insertQueue(queue);
22         }
23         // 4. 写内存
24         memoryDataCenter.insertQueue(queue);
25         System.out.println("[VirtualHost] 队列创建成功! queueName=" + queueName);
26         return true;
27     } catch (Exception e) {
28         System.out.println("[VirtualHost] 队列创建失败! queueName=" + queueName);

```

```

29         e.printStackTrace();
30         return false;
31     }
32 }

```

删除队列

```

1 // 删除队列
2 public boolean queueDelete(String queueName) {
3     // 真实的 queueName 需要拼接上 virtualHostName
4     queueName = virtualHostName + queueName;
5     try {
6         // 1. 根据 queueName 查询对应的队列对象
7         MSGQueue queue = memoryDataCenter.getQueue(queueName);
8         if (queue == null) {
9             throw new MqException("[VirtualHost] 队列不存在, 无法删除!");
10        }
11        // 2. 删除硬盘数据
12        if (queue.isDurable()) {
13            diskDataCenter.deleteQueue(queueName);
14        }
15        // 3. 删除内存数据
16        memoryDataCenter.deleteQueue(queueName);
17        System.out.println("[VirtualHost] 队列删除成功! queueName=" + queueName);
18        return true;
19    } catch (Exception e) {
20        System.out.println("[VirtualHost] 队列删除失败! queueName=" + queueName);
21        e.printStackTrace();
22        return false;
23    }
24 }

```

创建绑定

- bindingKey 是进行 topic 转发时的一个关键概念. 使用 router 类来检测是否是合法的 bindingKey.
- 后续再介绍 `router.checkBindingKeyValid` 的实现. 此处先留空.

```

1 // 创建绑定

```



```

2 public boolean queueBind(String queueName, String exchangeName, String
  bindingKey) {
3     // 真实的 queueName 需要拼接上 virtualHostName
4     queueName = virtualHostName + queueName;
5     exchangeName = virtualHostName + exchangeName;
6     try {
7         // 1. 判定 binding 是否存在
8         Binding existsBinding = memoryDataCenter.getBinding(queueName,
exchangeName);
9         if (existsBinding != null) {
10             throw new MqException("[VirtualHost] binding 已经存在! queueName="
+ queueName + ", exchangeName=" + exchangeName);
11         }
12         // 2. 校验 bindingKey 是否合法
13         if (!router.checkBindingKeyValid(bindingKey)) {
14             throw new MqException("[VirtualHost] bindingKey 非法! bindingKey="
+ bindingKey);
15         }
16         // 3. 创建 binding 对象
17         Binding binding = new Binding();
18         binding.setQueueName(queueName);
19         binding.setExchangeName(exchangeName);
20         binding.setBindingKey(bindingKey);
21         // 4. 获取到对应的 exchange 和 queue 对象
22         MSGQueue queue = memoryDataCenter.getQueue(queueName);
23         if (queue == null) {
24             throw new MqException("[VirtualHost] 对应的队列不存在! queueName=" +
queueName);
25         }
26         Exchange exchange = memoryDataCenter.getExchange(exchangeName);
27         if (exchange == null) {
28             throw new MqException("[VirtualHost] 对应的交换机不存在!
exchangeName=" + exchangeName);
29         }
30         // 5. 如果 exchange 和 queue 都是持久化的, 则 binding 也持久化.
31         if (queue.isDurable() && exchange.isDurable()) {
32             diskDataCenter.insertBinding(binding);
33         }
34         // 6. 写入内存
35         memoryDataCenter.insertBinding(binding);
36         System.out.println("[VirtualHost] 创建绑定成功! exchangeName=" +
exchangeName + ", queueName=" + queueName);
37         return true;
38     } catch (Exception e) {
39         System.out.println("[VirtualHost] 创建绑定失败! exchangeName=" +
exchangeName + ", queueName=" + queueName);
40         e.printStackTrace();

```

```
41         return false;
42     }
43 }
```

删除绑定

```
1 // 解除绑定
2 public boolean queueUnbind(String queueName, String exchangeName) {
3     // 真实的 queueName 需要拼接上 virtualHostName
4     queueName = virtualHostName + queueName;
5     exchangeName = virtualHostName + exchangeName;
6     try {
7         // 1. 获取到 binding
8         Binding binding = memoryDataCenter.getBinding(queueName, exchangeName);
9         if (binding == null) {
10             throw new Exception("[VirtualHost] 绑定不存在!");
11         }
12         // 2. 获取到对应的 exchange 和 queue 对象
13         MSGQueue queue = memoryDataCenter.getQueue(queueName);
14         if (queue == null) {
15             throw new Exception("[VirtualHost] 对应的队列不存在! queueName=" +
16 queueName);
17         }
18         Exchange exchange = memoryDataCenter.getExchange(exchangeName);
19         if (exchange == null) {
20             throw new Exception("[VirtualHost] 对应的交换机不存在! exchangeName="
21 + exchangeName);
22         }
23         // 3. 如果 exchange 和 queue 都是持久化的, 则 binding 从硬盘删除
24         if (queue.isDurable() && exchange.isDurable()) {
25             diskDataCenter.deleteBinding(binding);
26         }
27         // 4. 从内存删除 binding
28         memoryDataCenter.deleteBinding(binding);
29         System.out.println("[VirtualHost] 绑定删除成功! exchangeName=" +
30 exchangeName + ", queueName=" + queueName);
31         return true;
32     } catch (Exception e) {
33         System.out.println("[VirtualHost] 绑定删除失败! exchangeName=" +
34 exchangeName + ", queueName=" + queueName);
35         e.printStackTrace();
36         return false;
37     }
38 }
```

发布消息

- 发布消息其实是把消息发送给指定的 Exchange, 再根据 Exchange 和 Queue 的 Binding 关系, 转发到对应队列中.
- 发送消息需要指定 routingKey, 这个值的作用和 ExchangeType 是相关的.
 - Direct: routingKey 就是对应队列的名字. 此时不需要 binding 关系, 也不需要 bindingKey, 就可以直接转发消息.
 - Fanout: routingKey 不起作用, bindingKey 也不起作用. 此时消息会转发给绑定到该交换机上的所有队列中.
 - Topic: routingKey 是一个特定的字符串, 会和 bindingKey 进行匹配. 如果匹配成功, 则发到对应的队列中. 具体规则后续介绍.
- BasicProperties 是消息的元信息. body 是消息本体.

```

1 // 发送消息
2 public boolean basicPublish(String exchangeName, String routingKey,
3                             BasicProperties basicProperties, byte[] body) {
4     try {
5         // 1. 转换交换机名字. 如果是 null, 则使用默认交换机
6         if (exchangeName == null) {
7             exchangeName = "";
8         }
9         exchangeName = virtualhostName + exchangeName;
10
11         // 2. 检查参数合法性
12         if (!router.checkRoutingKeyValid(routingKey)) {
13             throw new MqException("[VirtualHost] routingKey 非法! routingKey="
14 + routingKey);
15         }
16
17         // 3. 查找到交换机对象
18         Exchange exchange = memoryDataCenter.getExchange(exchangeName);
19         if (exchange == null) {
20             throw new MqException("[VirtualHost] 交换机不存在! exchangeName=" +
21 exchangeName);
22         }
23
24         if (exchange.getType() == ExchangeType.DIRECT) {
25             String queueName = virtualhostName + routingKey;

```

```

24         // 4. 构造消息对象
25         Message message = Message.createMessageWithId(routingKey,
basicProperties, body);
26         // 5. 直接转发, 不需要 binding, 直接根据 routingKey 找到队列名, 进行转
发。
27         MSGQueue queue = memoryDataCenter.getQueue(queueName);
28         if (queue == null) {
29             throw new MqException("[VirtualHost] 队列不存在! queueName=" +
queueName);
30         }
31         // 6. 直接转发消息
32         sendMessage(queue, message);
33     } else {
34         // 4. 找到交换机对应的绑定对象
35         Map<String, Binding> bindings =
memoryDataCenter.getBindingsByExchange(exchangeName);
36
37         // 5. 遍历所有绑定, 进入消息转发逻辑。
38         for (Map.Entry<String, Binding> entry : bindings.entrySet()) {
39             // 1) 判定队列是否存在
40             Binding binding = entry.getValue();
41             MSGQueue queue =
memoryDataCenter.getQueue(binding.getQueueName());
42             if (queue == null) {
43                 throw new MqException("[VirtualHost] 队列不存在! queueName="
+ binding.getQueueName());
44             }
45             // 2) 构造消息对象。针对每次写入队列, 都构造一个唯一的消息对象 id。使
同一个消息, 在不同队列中也能有不同的消息 id。
46             // 如果两个队列中的消息 id 一样, 此时就可能在 messageMap 中只存在
一份消息, 而在 queueMessageMap 中存在多份消息。
47             // 此时针对消息进行消费操作, 就可能出现一个队列消费了之后, 把消息从
messageMap 删除了; 第二次再从另一个队列消费
48             // 的时候, 就无法从 messageMap 中获取到消息了。
49             Message message = Message.createMessageWithId(routingKey,
basicProperties, body);
50             // 3) 判定能否转发
51             if (!router.route(exchange.getType(), binding, message)) {
52                 continue;
53             }
54             // 4) 真正转发消息
55             sendMessage(queue, message);
56         }
57     }
58     return true;
59 } catch (Exception e) {
60     System.out.println("[VirtualHost] 消息发布失败!");

```

```

61         e.printStackTrace();
62         return false;
63     }
64 }

```

```

1 private void sendMessage(MSGQueue queue, Message message) throws Exception {
2     // 1. 先写硬盘
3     // deliverMode 为 1, 表示不持久化; 为 2 表示持久化. AMQP 协议规定的.
4     int deliveryMode = message.getBasicProperties().getDeliveryMode();
5     if (deliveryMode == 2) {
6         diskDataCenter.sendMessage(queue, message);
7     }
8     // 2. 再写内存
9     memoryDataCenter.sendMessage(queue, message);
10    // 3. 通知消费者去取消息
11    consumerManager.notifyConsume(queue.getName());
12 }

```

路由规则

实现 `mqserver.core.Router`

1) 实现 route 方法

```

1 public class Router {
2     public boolean route(ExchangeType exchangeType, Binding binding, Message
        message) throws MqException {
3         // 根据不同的 exchangeType 进行不同的转发逻辑
4         // DIRECT 的转发逻辑已经在外部判定过.
5         if (exchangeType == ExchangeType.FANOUT) {
6             return routeFanout(binding, message);
7         } else if (exchangeType == ExchangeType.TOPIC) {
8             return routeTopic(binding, message);
9         } else {
10            throw new MqException("[VirtualHost] 未知的 exchangeType!
                exchangeType=" + exchangeType);
11        }
12    }
13
14    private boolean routeFanout(Binding binding, Message message) {

```

```
15      // 对于 fanout 类型, 直接转发, 不需要进行任何匹配.
16      return true;
17  }
18 }
```

2) 实现 checkRoutingKeyValid

一个 RoutingKey 是由数字字母下划线构成的, 并且可以使用 . 分成若干部分.

形如 `aaa.bbb.ccc`

```
1  // 不包含通配符, 规则更简单.
2  public boolean checkRoutingKeyValid(String routingKey) {
3      if (routingKey.length() == 0) {
4          return true;
5      }
6      // 数字字母下划线构成
7      for (int i = 0; i < routingKey.length(); i++) {
8          char ch = routingKey.charAt(i);
9          if (ch >= 'A' && ch <= 'Z') {
10             continue;
11         }
12         if (ch >= 'a' && ch <= 'z') {
13             continue;
14         }
15         if (ch >= '0' && ch <= '9') {
16             continue;
17         }
18         if (ch == '_' || ch == '.') {
19             continue;
20         }
21         return false;
22     }
23     return true;
24 }
```

3) 实现 checkBindingKeyValid

一个 BindingKey 是由数字字母下划线构成的, 并且使用 . 分成若干部分.

另外, 支持 * 和 # 两种通配符. (* # 只能作为 . 切分出来的独立部分, 不能和其他数字字母混用, 比如 a*.b 是合法的, a.*b 是不合法的).

其中 * 可以匹配任意一个单词.

其中 # 可以匹配任意零个或者多个单词.

例如:

bindingKey 为 a*.b, 可以匹配 routingKey 为 a.a.b 和 a.b.b 和 a.aaa.b

bindingKey 为 a#.b, 可以匹配 routingKey 为 a.a.b 和 a.b.b 和 a.aaa.b 和 a.aa.bb.b 和 a.b

```
1 // 需要考虑通配符, 复杂一些
2 public boolean checkBindingValid(String bindingKey) {
3     // 1. 允许是空字符串
4     // 2. 数字字母下划线构成
5     // 3. 可以包含通配符
6     // 4. # 不能连续出现.
7     // 5. # 和 * 不能相邻
8     if (bindingKey.length() == 0) {
9         return true;
10    }
11    // 先判定基础构成
12    for (int i = 0; i < bindingKey.length(); i++) {
13        char ch = bindingKey.charAt(i);
14        if (ch >= 'A' && ch <= 'Z') {
15            continue;
16        }
17        if (ch >= 'a' && ch <= 'z') {
18            continue;
19        }
20        if (ch >= '0' && ch <= '9') {
21            continue;
22        }
23        if (ch == '.' || ch == '_' || ch == '*' || ch == '#') {
24            continue;
25        }
26        return false;
27    }
28    // 再判定每个词的情况
29    // 比如 aaa.a*a 这种应该视为非法.
30    String[] words = bindingKey.split("\\.");
31    for (String word : words) {
32        if (word.length() > 1 && (word.contains("*") ||
33            word.contains("#"))) {
34            return false;
35        }
36    }
37    // 再判定相邻词的情况
38    for (int i = 0; i < words.length - 1; i++) {
```



```

38         // 连续两个 ##
39         if (words[i].equals("#") && words[i + 1].equals("#")) {
40             return false;
41         }
42         // # 连着 *
43         if (words[i].equals("#") && words[i + 1].equals("*")) {
44             return false;
45         }
46         // * 连着 #
47         if (words[i].equals("*") && words[i + 1].equals("#")) {
48             return false;
49         }
50     }
51     return true;
52 }

```

4) 实现 routeTopic

```

1  // 需要按照通配符匹配
2  // binding key 包含通配符
3  // 1. * 表示任意一个 token 都可以匹配
4  // 2. # 表示任意 0 个或 N 个 token 都可以匹配
5  // 3. 其他内容则要求严格匹配。
6  // 4. # 不会连续出现。# 和 * 不会相邻。
7  // routing key 不包含通配符。这个在发消息的时候校验
8  private boolean routeTopic(Binding binding, Message message) {
9      // 按照 . 来切分 binding key 和 routing key
10     String[] bindingTokens = binding.getBindingKey().split("\\.");
11     String[] routingTokens = message.getRoutingKey().split("\\.");
12     // 使用双指针的方式来实现匹配
13     // 1. 如果是普通字符，直接匹配内容是否相等，不相等则返回 false，相等直接进入下一轮
14     // 2. 如果是 *，直接进入下一轮
15     // 3. 如果 # 没有下一个位置，则直接返回 true
16     // 4. 如果遇到 #，则找到 # 下一个位置的 token 在 routingKey 中的位置。
17     // 5. 如果能找到对应的位置了，就可以继续匹配。如果找不到，就返回 false
18     // 6. 循环结束后，检查两个下标是否同时到达末尾。是则匹配成功，否则匹配失败。
19     int bindingIndex = 0;
20     int routingIndex = 0;
21     while (bindingIndex < bindingTokens.length && routingIndex <
routingTokens.length) {
22         if (bindingTokens[bindingIndex].equals("*")) {
23             // 2. 如果是 *，直接进入下一轮
24             // 直接进入下一轮比较

```



```

25         bindingIndex++;
26         routingIndex++;
27     } else if (bindingTokens[bindingIndex].equals("#")) {
28         bindingIndex++;
29         if (bindingIndex == bindingTokens.length) {
30             // 3. 如果 # 没有下一个位置, 则直接返回 true
31             return true;
32         }
33         // 4. 如果遇到 # , 则找到 # 下一个位置的 token 在 routingKey 中的位置.
34         routingIndex = findNextMatch(routingTokens, routingIndex,
bindingTokens[bindingIndex]);
35         // 5. 如果能找到对应的位置了, 就可以继续下一轮匹配. 如果找不到, 就返回
false
36         if (routingIndex == -1) {
37             return false;
38         }
39         bindingIndex++;
40         routingIndex++;
41     } else {
42         // 1. 如果是普通字符, 直接匹配内容是否相等, 不相等则返回 false, 相等直接进
入下一轮
43         if
(!bindingTokens[bindingIndex].equals(routingTokens[routingIndex])) {
44             return false;
45         }
46         bindingIndex++;
47         routingIndex++;
48     }
49 }
50
51 // 如果两方不能同时结束, 则也视为匹配失败.
52 // 比如 aaa.*.bbb 和 aaa.bbb
53 if (bindingIndex == bindingTokens.length && routingIndex ==
routingTokens.length) {
54     return true;
55 }
56 return false;
57 }
58
59 private int findNextMatch(String[] routingTokens, int routingIndex, String
bindingToken) {
60     for (int i = routingIndex; i < routingTokens.length; i++) {
61         if (routingTokens[i].equals(bindingToken)) {
62             return i;
63         }
64     }
65     return -1;

```

5) 匹配规则测试用例

	routing key	result
1 // [测试用例]		
2 // binding key		
3 // aaa	aaa	true
4 // aaa.bbb	aaa.bbb	true
5 // aaa.bbb	aaa.bbb.ccc	false
6 // aaa.bbb	aaa.ccc	false
7 // aaa.bbb.ccc	aaa.bbb.ccc	true
8 // aaa.*	aaa.bbb	true
9 // aaa.*.bbb	aaa.bbb.ccc	false
10 // *.aaa.bbb	aaa.bbb	false
11 // #	aaa.bbb.ccc	true
12 // aaa.#	aaa.bbb	true
13 // aaa.#	aaa.bbb.ccc	true
14 // aaa.#.ccc	aaa.ccc	true
15 // aaa.#.ccc	aaa.bbb.ccc	true
16 // aaa.#.ccc	aaa.aaa.bbb.ccc	true
17 // #.ccc	ccc	true
18 // #.ccc	aaa.bbb.ccc	true

6) 测试 Router

创建 RouterTests

```

1 @SpringBootTest
2 public class RouterTests {
3     private Router router = new Router();
4     private Message message = null;
5     private Binding binding = null;
6
7     @BeforeEach
8     public void setUp() {
9         message = new Message();
10        binding = new Binding();
11    }
12
13    @AfterEach

```

```
14     public void tearDown() {
15         message = null;
16         binding = null;
17     }
18 }
```

```
1 @Test
2 public void test() throws MqException {
3     binding.setBindingKey("aaa");
4     message.setRoutingKey("aaa");
5     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
6
7     binding.setBindingKey("aaa.bbb");
8     message.setRoutingKey("aaa.bbb");
9     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
10
11     binding.setBindingKey("aaa.bbb");
12     message.setRoutingKey("aaa.bbb.ccc");
13     Assertions.assertFalse(router.route(ExchangeType.TOPIC, binding, message));
14
15     binding.setBindingKey("aaa.bbb");
16     message.setRoutingKey("aaa.ccc");
17     Assertions.assertFalse(router.route(ExchangeType.TOPIC, binding, message));
18
19     binding.setBindingKey("aaa.bbb.ccc");
20     message.setRoutingKey("aaa.bbb.ccc");
21     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
22
23     binding.setBindingKey("aaa.*");
24     message.setRoutingKey("aaa.bbb");
25     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
26
27     binding.setBindingKey("aaa.*.bbb");
28     message.setRoutingKey("aaa.bbb.ccc");
29     Assertions.assertFalse(router.route(ExchangeType.TOPIC, binding, message));
30
31     binding.setBindingKey("*.aaa.bbb");
32     message.setRoutingKey("aaa.bbb");
33     Assertions.assertFalse(router.route(ExchangeType.TOPIC, binding, message));
34
35     binding.setBindingKey("#");
36     message.setRoutingKey("aaa.bbb.ccc");
37     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
38
39     binding.setBindingKey("aaa.#");
```

```

40     message.setRoutingKey("aaa.bbb");
41     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
42
43     binding.setBindingKey("aaa.#");
44     message.setRoutingKey("aaa.bbb.ccc");
45     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
46
47     binding.setBindingKey("aaa.#.ccc");
48     message.setRoutingKey("aaa.ccc");
49     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
50
51     binding.setBindingKey("aaa.#.ccc");
52     message.setRoutingKey("aaa.bbb.ccc");
53     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
54
55     binding.setBindingKey("aaa.#.ccc");
56     message.setRoutingKey("aaa.aaa.bbb.ccc");
57     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
58
59     binding.setBindingKey("#.ccc");
60     message.setRoutingKey("ccc");
61     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
62
63     binding.setBindingKey("#.ccc");
64     message.setRoutingKey("aaa.bbb.ccc");
65     Assertions.assertTrue(router.route(ExchangeType.TOPIC, binding, message));
66 }

```

订阅消息

1) 添加一个订阅者

```

1 // 订阅消息
2 // 如果是多个消费者消费一个队列，将使用轮询的方式进行消费。
3 // 参数的 consumerTag 应该在网络通信部分设定。
4 public boolean basicConsume(String consumerTag, String queueName, boolean
    autoAck, Consumer consumer) {
5     queueName = virtualHostName + queueName;
6     try {
7         // 把 consumer 加到监听线程管理的消费者数组中
8         consumerManager.addConsumer(consumerTag, queueName, autoAck, consumer);
9         System.out.println("[VirtualHost] basicConsume 成功! queueName=" +
    queueName);

```

```

10         return true;
11     } catch (Exception e) {
12         System.out.println("[VirtualHost] basicConsume 失败! queueName=" +
            queueName);
13         e.printStackTrace();
14         return false;
15     }
16 }

```

Consumer 相当于一个回调函数. 放到 `common.Consumer` 中.

```

1 @FunctionalInterface
2 public interface Consumer {
3     // consumerTag 消费者标识, 后面使用 channelId 填充.
4     void handleDelivery(String consumerTag, BasicProperties properties, byte[]
        body) throws MqException, IOException;
5 }

```

2) 创建订阅者管理管理类

创建 `mqserver.core.ConsumerManager`

```

1 public class ConsumerManager {
2     private VirtualHost parent;
3     // 存放令牌的队列. 通过令牌来触发消费线程的消费操作.
4     private BlockingQueue<String> tokenQueue = new LinkedBlockingQueue<>();
5     private ExecutorService workerPool = Executors.newFixedThreadPool(4);
6 }

```

- parent 用来记录虚拟主机.
- 使用一个阻塞队列用来触发消息消费. 称为令牌队列. 每次有消息过来了, 都往队列中放一个令牌(也就是队列名), 然后消费者再去消费对应队列的消息.
- 使用一个线程池用来执行消息回调.

这样令牌队列的设定避免搞出来太多线程. 否则就需要给每个队列都安排一个单独的线程了, 如果队列很多则开销就比较大了.

3) 添加令牌接口

```

1 // 通知消费者去消费消息
2 public void notifyConsume(String queueName) throws InterruptedException {
3     tokenQueue.put(queueName);
4 }

```

4) 实现添加订阅者

- 新来订阅者的时候, 需要先消费掉之前积压的消息.
- consumeMessage 真正的消息消费操作, 一会再实现.

```

1 public void addConsumer(String consumerTag, String queueName, boolean autoAck,
2     Consumer consumer) throws MqException {
3     // 消费已经积压的消息消息
4     MSGQueue msgQueue = parent.getMemoryDataCenter().getQueue(queueName);
5     if (msgQueue == null) {
6         throw new MqException("[ConsumerManager] 队列不存在! queueName=" +
7             queueName);
8     }
9     ConsumerEnv consumerEnv = new ConsumerEnv(consumerTag, queueName, autoAck,
10         consumer);
11     synchronized (msgQueue) {
12         msgQueue.addConsumerEnv(consumerEnv);
13     }
14     // 把已经积压的 n 个数据都先消费掉
15     int n = parent.getMemoryDataCenter().getMessageCount(queueName);
16     for (int i = 0; i < n; i++) {
17         consumeMessage(msgQueue);
18     }
19 }

```

创建 `ConsumerEnv`, 这个类表示一个订阅者的执行环境.

```

1 // 表示一个消费者的上下文环境
2 public class ConsumerEnv {
3     private String consumerTag;
4     private String queueName;
5     private boolean autoAck;

```

```

6     private Consumer consumer;
7
8     public ConsumerEnv(String consumerTag, String queueName, boolean autoAck,
        Consumer consumer) {
9         this.consumerTag = consumerTag;
10        this.queueName = queueName;
11        this.autoAck = autoAck;
12        this.consumer = consumer;
13    }
14
15    // 省略 getter setter
16 }

```

给 `MsgQueue` 添加一个订阅者列表.

```

1 // 该队列被哪些消费者订阅
2 private List<ConsumerEnv> consumerEnvList = new ArrayList<>();
3 // 轮询序号
4 private AtomicInteger consumerSeq = new AtomicInteger(0);
5
6 public void addConsumerEnv(ConsumerEnv consumerEnv) {
7     consumerEnvList.add(consumerEnv);
8 }
9
10 public ConsumerEnv chooseConsumer() {
11     if (consumerEnvList.size() == 0) {
12         return null;
13     }
14     int index = consumerSeq.get() % consumerEnvList.size();
15     consumerSeq.getAndIncrement();
16     return consumerEnvList.get(index);
17 }

```

此处的 `chooseConsumer` 是实现一个轮询效果. 如果一个队列有多个订阅者, 将会按照轮询的方式轮流拿到消息.

5) 实现扫描线程

在 `ConsumerManager` 中创建一个线程, 不停的尝试扫描令牌队列. 如果拿到了令牌, 就真正触发消费消息操作.

```

1 public ConsumerManager(VirtualHost parent) {
2     this.parent = parent;
3
4     // 启动扫描线程
5     Thread scanThread = new Thread(() -> {
6         while (true) {
7             try {
8                 // 1. 拿到令牌
9                 String queueName = tokenQueue.take();
10                // 2. 找到队列
11                MSGQueue msgQueue =
parent.getMemoryDataCenter().getQueue(queueName);
12                if (msgQueue == null) {
13                    throw new MqException("[ConsumerManager] 队列不存在!
queueName=" + queueName);
14                }
15                // 3. 消费一个数据
16                synchronized (msgQueue) {
17                    consumeMessage(msgQueue);
18                }
19            } catch (MqException | InterruptedException e) {
20                e.printStackTrace();
21            }
22        }
23    }, "scanThread");
24    scanThread.start();
25 }

```

6) 实现消费消息

所谓的消费消息, 其实就是调用消息的回调. 并把消息删除掉.

```

1 private void consumeMessage(MSGQueue msgQueue) throws MqException {
2     // 1. 按照轮询方式, 先找个消费者出来
3     ConsumerEnv luckyDog = msgQueue.chooseConsumer();
4     if (luckyDog == null) {
5         // 如果当前还没有订阅者, 就先暂时不消费.
6         return;
7     }
8     // 2. 从指定队列中取一个元素
9     Message message =
parent.getMemoryDataCenter().pollMessage(msgQueue.getName());
10    if (message == null) {
11        return;

```



```

12     }
13     System.out.println("[ConsumerManager] 消息被成功消费! queueName=" +
msgQueue.getName() + ", messageId=" + message.getMessageId());
14     // 3. 丢到线程池中干活。回调执行时间可能比较长。不适合让扫描线程去调用。
15     workerPool.submit(() -> {
16         try {
17             // 1. 先把消息放到待确认队列中
18             // (这个逻辑必须放到执行回调前面。如果是 autoAck false, 在回调内部会调
用 basicAck, 执行彻底删除。需要先放到待确认队列, 才能彻底删除)
19             parent.getMemoryDataCenter().addMessageWaitAck(msgQueue.getName(),
message);
20             // 2. 调用消费者的回调。如果回调抛出异常了, 则不会对消息进行任何 ack 操作。
21             // 相当于消息仍然处在待消费的状态。
22             luckyDog.getConsumer().handleDelivery(luckyDog.getConsumerTag(),
message.getBasicProperties(), message.getBody());
23             // 3. 如果消息是自动确认, 则可以直接把消息彻底删除了。
24             // (这个逻辑必须放到执行回调后面。万一执行回调一半服务器崩溃, 这个消息仍
然存在于硬盘上, 下次启动还可以被继续消费到)
25             if (luckyDog.isAutoAck()) {
26                 // 则修改硬盘上的消息为 "无效"。同时删除内存中的消息
27                 if (message.getDeliveryMode() == 2) {
28                     parent.getDiskDataCenter().deleteMessage(msgQueue,
message);
29                 }
30                 parent.getMemoryDataCenter().removeMessageWaitAck(msgQueue.getName(),
message.getMessageId());
31                 parent.getMemoryDataCenter().removeMessage(message.getMessageId());
32             }
33         } catch (MqException | IOException | ClassNotFoundException e) {
34             e.printStackTrace();
35         }
36     });
37 }

```

注意: 一个队列可能有 N 个消费者, 此处应该按照轮询的方式挑一个消费者进行消费。

小结

一. 消费消息的两种典型情况

1) 订阅者已经存在了, 才发送消息

这种直接获取队列的订阅者, 从中按照轮询的方式挑一个消费者来调用回调即可。

2) 消息先发送到队列了, 订阅者还没到.

此时当订阅者到达, 就快速把指定队列中的消息全都消费掉.

二. 关于消息不丢失的论证

每个消息在从内存队列中出队列时, 都会先进入 待确认 中.

- 如果 autoAck 为 true

消息被消费完毕后(执行完消息回调之后), 再执行清除工作.

分别清除硬盘数据, 待确认队列, 消息中心.

- 如果 autoAck 为 false

在回调内部, 进行清除工作.

分别清除硬盘数据, 待确认队列, 消息中心.

1) 执行消息回调的时候抛出异常

此时消息仍然处在待确认队列中.

此时可以用一个线程扫描待确认队列, 如果发现队列中的消息超时未确认, 则放入死信队列.

死信队列咱们此处暂不实现.

2) 执行消息回调的时候服务器宕机

内存所有数据都没了, 但是消息在硬盘上仍然存在. 会在服务下次启动的时候, 加载回内存. 重新被消费到.

消息确认

下列方法只是手动应答的时候才会使用.

应答成功, 则把消息删除掉.

```
1 public boolean basicAck(String queueName, String messageId) {
2     queueName = virtualHostName + queueName;
3     try {
4         // 删除待 ack 队列中的数据
5         memoryDataCenter.removeMessageWaitAck(queueName, messageId);
6         // 删除硬盘上的数据
7         MSGQueue queue = memoryDataCenter.getQueue(queueName);
```

```

8      Message message = memoryDataCenter.getMessage(messageId);
9      if (message.getDeliveryMode() == 2) {
10         diskDataCenter.deleteMessage(queue, message);
11     }
12     // 删除内存中的数据
13     memoryDataCenter.removeMessage(messageId);
14     System.out.println("[VirtualHost] basicAck 成功! queueName=" +
        queueName + ", messageId=" + messageId);
15     return true;
16 } catch (Exception e) {
17     System.out.println("[VirtualHost] basicAck 失败! queueName=" +
        queueName + ", messageId=" + messageId);
18     e.printStackTrace();
19 }
20 return false;
21 }

```

对于 RabbitMQ 来说, 还支持否定应答的情况. 此处没有支持. 同学们可以自行尝试实现.

测试 VirtualHost

编写 `VirtualHostTests`

- 操作数据库, 需要先启动 Spring 服务.
- 同时, 需要先关闭 Spring 服务, 才能删除数据库文件
- 使用 `FileUtils.deleteDirectory` 递归的删除目录中的内容. 这个是 Spring 自带的类 `org.apache.tomcat.util.http.fileupload.FileUtils`

```

1 @SpringBootTest
2 public class VirtualHostTests {
3     private VirtualHost virtualHost = null;
4
5     @BeforeEach
6     public void setUp() {
7         JavaMessageQueueApplication.ac =
            SpringApplication.run(JavaMessageQueueApplication.class);
8         virtualHost = new VirtualHost("");
9     }
10
11     @AfterEach
12     public void tearDown() throws IOException {
13         JavaMessageQueueApplication.ac.close();
14         File dbFile = new File("meta.db");

```

```
15         dbFile.delete();
16         File dataFile = new File("./data");
17         FileUtils.deleteDirectory(dataFile);
18     }
19 }
```

编写测试用例

```
1  @Test
2  public void testExchangeDeclare() {
3      boolean ok = virtualHost.exchangeDeclare("testExchange",
4      ExchangeType.DIRECT, true, false, null);
5      Assertions.assertTrue(ok);
6  }
7  @Test
8  public void testExchangeDelete() {
9      boolean ok = virtualHost.exchangeDeclare("testExchange",
10      ExchangeType.DIRECT, true, false, null);
11      Assertions.assertTrue(ok);
12      ok = virtualHost.exchangeDelete("testExchange");
13      Assertions.assertTrue(ok);
14  }
15  @Test
16  public void testQueueDeclare() {
17      boolean ok = virtualHost.queueDeclare("testQueue", true, false, false,
18      null);
19      Assertions.assertTrue(ok);
20  }
21  @Test
22  public void testQueueDelete() {
23      boolean ok = virtualHost.queueDeclare("testQueue", true, false, false,
24      null);
25      Assertions.assertTrue(ok);
26      ok = virtualHost.queueDelete("testQueue");
27      Assertions.assertTrue(ok);
28  }
29  @Test
30  public void testQueueBind() {
31      boolean ok = virtualHost.queueDeclare("testQueue", true, false, false,
32      null);
33      Assertions.assertTrue(ok);
34  }
```

```
33     ok = virtualHost.exchangeDeclare("testExchange", ExchangeType.DIRECT,
    true, false, null);
34     Assertions.assertTrue(ok);
35     ok = virtualHost.queueBind("testQueue", "testExchange", "testBindingKey");
36     Assertions.assertTrue(ok);
37 }
38
39 @Test
40 public void testQueueUnbind() {
41     boolean ok = virtualHost.queueDeclare("testQueue", true, false, false,
    null);
42     Assertions.assertTrue(ok);
43     ok = virtualHost.exchangeDeclare("testExchange", ExchangeType.DIRECT,
    true, false, null);
44     Assertions.assertTrue(ok);
45     ok = virtualHost.queueBind("testQueue", "testExchange", "testBindingKey");
46     Assertions.assertTrue(ok);
47     ok = virtualHost.queueUnbind("testQueue", "testExchange");
48     Assertions.assertTrue(ok);
49 }
50
51 @Test
52 public void testBasicPublic() {
53     boolean ok = virtualHost.queueDeclare("testQueue", true, false, false,
    null);
54     Assertions.assertTrue(ok);
55     ok = virtualHost.exchangeDeclare("testExchange", ExchangeType.DIRECT,
    true, false, null);
56     Assertions.assertTrue(ok);
57
58     ok = virtualHost.basicPublish("testExchange", "testQueue", null,
    "hello".getBytes());
59     Assertions.assertTrue(ok);
60 }
61
62 // 先订阅消息，后发送消息
63 @Test
64 public void testBasicConsumeDirect1() throws InterruptedException {
65     boolean ok = virtualHost.queueDeclare("testQueue", true, false, false,
    null);
66     Assertions.assertTrue(ok);
67     ok = virtualHost.exchangeDeclare("testExchange", ExchangeType.DIRECT,
    true, false, null);
68     Assertions.assertTrue(ok);
69
70     ok = virtualHost.basicConsume("testConsumerTag", "testQueue", true, new
    Consumer() {
```

```

71     @Override
72     public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] body) throws MqException, IOException {
73         System.out.println("messageId=" + properties.getMessageId());
74         Assertions.assertEquals("testQueue", properties.getRoutingKey());
75         Assertions.assertEquals(1, properties.getDeliveryMode());
76         Assertions.assertArrayEquals("hello".getBytes(), body);
77     }
78 });
79 Assertions.assertTrue(ok);
80
81     ok = virtualHost.basicPublish("testExchange", "testQueue", null,
"hello".getBytes());
82     Assertions.assertTrue(ok);
83 }
84
85 // 先发送消息, 后订阅
86 @Test
87 public void testBasicConsumeDirect2() throws InterruptedException {
88     boolean ok = virtualHost.queueDeclare("testQueue", true, false, false,
null);
89     Assertions.assertTrue(ok);
90     ok = virtualHost.exchangeDeclare("testExchange", ExchangeType.DIRECT,
true, false, null);
91     Assertions.assertTrue(ok);
92
93     ok = virtualHost.basicPublish("testExchange", "testQueue", null,
"hello".getBytes());
94     Assertions.assertTrue(ok);
95
96     ok = virtualHost.basicConsume("testConsumerTag", "testQueue", true, new
Consumer() {
97         @Override
98         public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] body) throws MqException, IOException {
99             System.out.println("messageId=" + properties.getMessageId());
100             Assertions.assertEquals("testQueue", properties.getRoutingKey());
101             Assertions.assertEquals(1, properties.getDeliveryMode());
102             Assertions.assertArrayEquals("hello".getBytes(), body);
103         }
104     });
105     Assertions.assertTrue(ok);
106     // 保证消费者有足够的时间完成消费
107     Thread.sleep(500);
108 }
109
110 @Test

```

```

111 public void testBasicConsumeFanout() throws InterruptedException {
112     boolean ok = virtualHost.exchangeDeclare("testExchange",
ExchangeType.FANOUT, true, false, null);
113     Assertions.assertTrue(ok);
114
115     ok = virtualHost.queueDeclare("testQueue1", true, false, false, null);
116     Assertions.assertTrue(ok);
117     ok = virtualHost.queueBind("testQueue1", "testExchange", "");
118     Assertions.assertTrue(ok);
119     ok = virtualHost.queueDeclare("testQueue2", true, false, false, null);
120     Assertions.assertTrue(ok);
121     ok = virtualHost.queueBind("testQueue2", "testExchange", "");
122     Assertions.assertTrue(ok);
123
124     ok = virtualHost.basicPublish("testExchange", "", null,
"hello".getBytes());
125     Assertions.assertTrue(ok);
126
127     ok = virtualHost.basicConsume("testConsumerTag", "testQueue1", false, new
Consumer() {
128         @Override
129         public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] body) throws MqException, IOException {
130             System.out.println("messageId=" + properties.getMessageId());
131             Assertions.assertEquals("testQueue1", properties.getRoutingKey());
132             Assertions.assertEquals(1, properties.getDeliveryMode());
133             Assertions.assertArrayEquals("hello".getBytes(), body);
134         }
135     });
136     Assertions.assertTrue(ok);
137
138     ok = virtualHost.basicConsume("testConsumerTag", "testQueue2", true, new
Consumer() {
139         @Override
140         public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] body) throws MqException, IOException {
141             System.out.println("messageId=" + properties.getMessageId());
142             Assertions.assertEquals("testQueue2", properties.getRoutingKey());
143             Assertions.assertEquals(1, properties.getDeliveryMode());
144             Assertions.assertArrayEquals("hello".getBytes(), body);
145         }
146     });
147     Assertions.assertTrue(ok);
148
149     Thread.sleep(500);
150 }
151

```



```

152 @Test
153 public void testBasicConsumeTopic() throws InterruptedException {
154     boolean ok = virtualHost.exchangeDeclare("testExchange",
ExchangeType.TOPIC, true, false, null);
155     Assertions.assertTrue(ok);
156     ok = virtualHost.queueDeclare("testQueue", true, false, false, null);
157     Assertions.assertTrue(ok);
158     ok = virtualHost.queueBind("testQueue", "testExchange", "aaa.*");
159     Assertions.assertTrue(ok);
160
161     ok = virtualHost.basicPublish("testExchange", "aaa.bbb", null,
"hello".getBytes());
162     Assertions.assertTrue(ok);
163
164     ok = virtualHost.basicConsume("testConsumerTag", "testQueue", true, new
Consumer() {
165         @Override
166         public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] body) throws MqException, IOException {
167             System.out.println("messageId=" + properties.getMessageId());
168             Assertions.assertEquals("testQueue", properties.getRoutingKey());
169             Assertions.assertEquals(1, properties.getDeliveryMode());
170             Assertions.assertArrayEquals("hello".getBytes(), body);
171         }
172     });
173     Assertions.assertTrue(ok);
174
175     Thread.sleep(500);
176 }
177
178 @Test
179 public void testBasicAck() throws InterruptedException {
180     boolean ok = virtualHost.queueDeclare("testQueue", true, false, false,
null);
181     Assertions.assertTrue(ok);
182     ok = virtualHost.exchangeDeclare("testExchange", ExchangeType.DIRECT,
true, false, null);
183     Assertions.assertTrue(ok);
184
185     ok = virtualHost.basicPublish("testExchange", "testQueue", null,
"hello".getBytes());
186     Assertions.assertTrue(ok);
187
188     Thread.sleep(500);
189
190     ok = virtualHost.basicConsume("testConsumerTag", "testQueue", false, new
Consumer() {

```



```

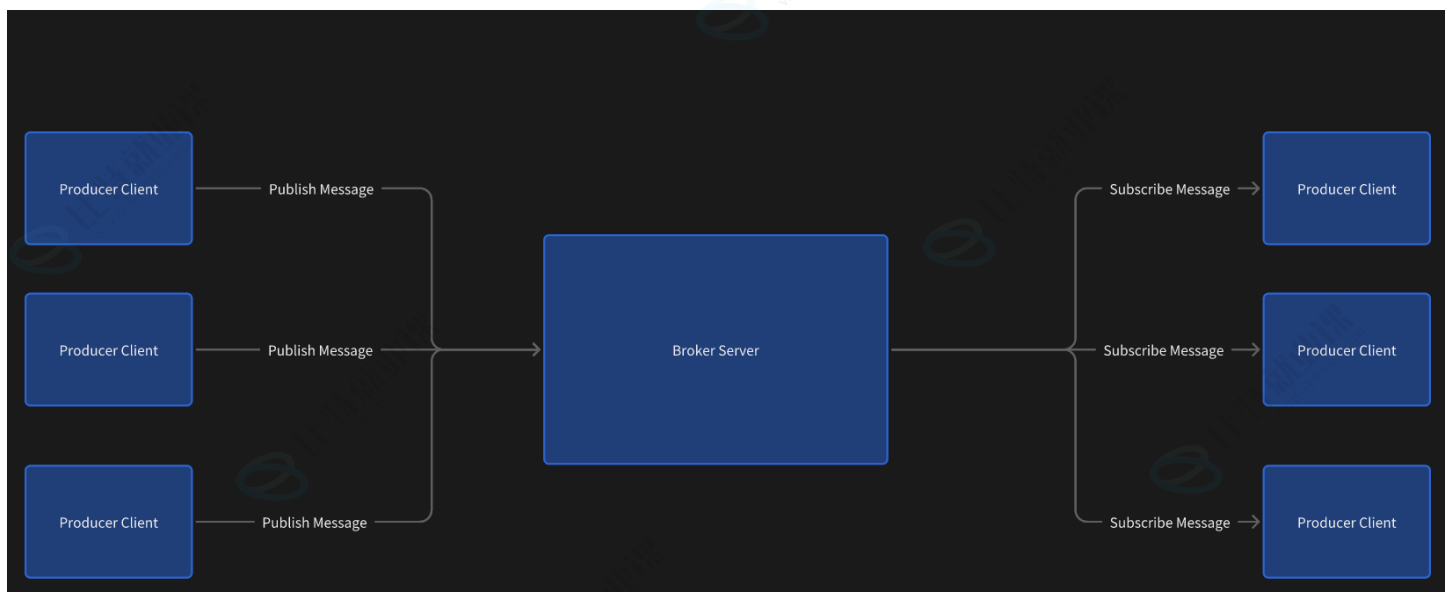
191     @Override
192     public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] body) throws MqException, IOException {
193         System.out.println("messageId=" + properties.getMessageId());
194         Assertions.assertEquals("testQueue", properties.getRoutingKey());
195         Assertions.assertEquals(1, properties.getDeliveryMode());
196         Assertions.assertArrayEquals("hello".getBytes(), body);
197         System.out.println("=====");
198         // 手动调用 ack
199         Assertions.assertTrue(virtualHost.basicAck("testQueue",
properties.getMessageId()));
200     }
201 });
202 Assertions.assertTrue(ok);
203
204 Thread.sleep(500);
205 }

```

十一. 网络通信协议设计

明确需求

接下来需要考虑客户端和服务端之间的通信. 回顾交互模型.



生产者和消费者都是客户端, 都需要通过网络和 Broker Server 进行通信.

此处我们使用 TCP 协议, 来作为通信的底层协议. 同时在这个基础上自定义应用层协议, 完成客户端对服务器这边功能的远程调用.

要调用的功能有:

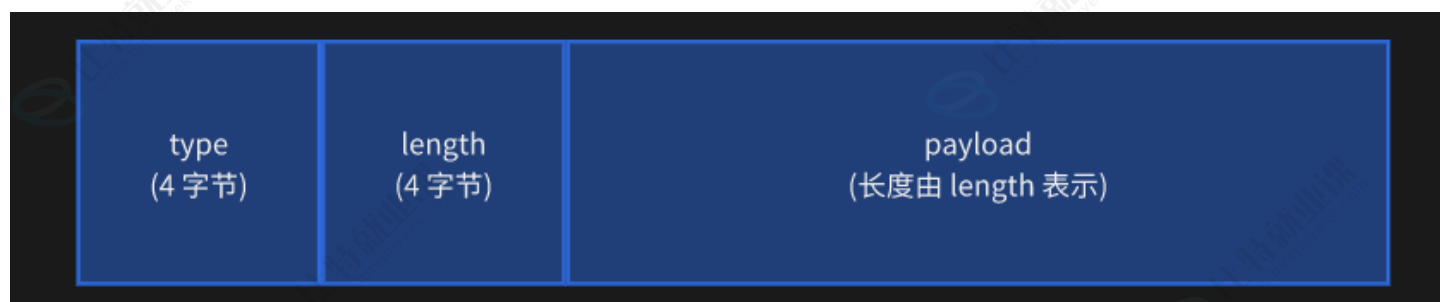
- 创建 channel
- 关闭 channel
- 创建 exchange
- 删除 exchange
- 创建 queue
- 删除 queue
- 创建 binding
- 删除 binding
- 发送 message
- 订阅 message
- 发送 ack
- 返回 message (服务器 -> 客户端)

设计应用层协议

使用二进制的方式设定协议.

因为 Message 的消息体本身就是二进制的. 因此不太方便使用 json 等文本格式的协议.

请求:



响应:



其中 type 表示请求响应不同的功能. 取值如下:

- 0x1 创建 channel
- 0x2 关闭 channel
- 0x3 创建 exchange
- 0x4 销毁 exchange
- 0x5 创建 queue
- 0x6 销毁 queue
- 0x7 创建 binding
- 0x8 销毁 binding
- 0x9 发送 message
- 0xa 订阅 message
- 0xb 返回 ack
- 0xc 服务器给客户端推送的消息. (被订阅的消息) 响应独有的.

其中 payload 部分, 会根据不同的 type, 存在不同的格式.

对于请求来说, payload 表示这次方法调用的各种参数信息.

对于响应来说, payload 表示这次方法调用的返回值.

定义 Request / Response

创建 `common.Request`

```
1 public class Request {  
2     private int type;  
3     private int length;  
4     private byte[] payload;  
5     // 省略 getter setter  
6 }
```

创建 `common.Response`

```
1 public class Response {
```

```
2     private int type;
3     private int length;
4     private byte[] payload;
5     // 省略 getter setter
6 }
```

定义参数父类

构造一个类表示方法的参数, 作为 Request 的 payload.

不同的方法中, 参数形态各异, 但是有些信息是通用的, 使用一个父类表示出来. 具体每个方法的参数再通过继承的方式体现.

common.BasicArguments

```
1 public class BaseArguments implements Serializable {
2     // 表示一次请求/响应的唯一 id. 用来把响应和请求对上.
3     protected String rid;
4     protected String channelId;
5
6     // 省略 getter setter
7 }
```

- 此处的 rid 和 channelId 都是基于 UUID 来生成的. rid 用来标识一个请求-响应. 这一点在请求响应比较多时非常重要.

定义返回值父类

和参数同理, 也需要构造一个类表示返回值, 作为 Response 的 payload.

common.BasicReturns

```
1 public class BaseReturns implements Serializable {
2     // 表示一次请求/响应的唯一 id. 用来把响应和请求对上.
3     protected String rid;
4     protected String channelId;
5     protected boolean ok;
6
7     // 省略 getter setter
8 }
```

定义其他参数类

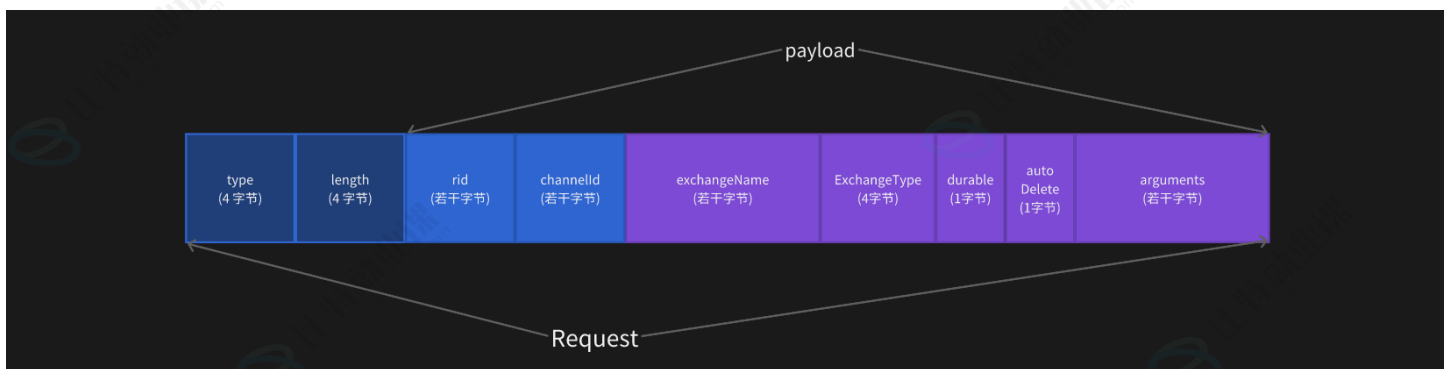
针对每个 VirtualHost 提供的方法, 都需要有一个类表示对应的参数.

1) ExchangeDeclareArguments

```
1 public class ExchangeDeclareArguments extends BaseArguments implements
  Serializable {
2     private String exchangeName;
3     private ExchangeType exchangeType;
4     private boolean durable;
5     private boolean autoDelete;
6     private Map<String, Object> arguments;
7 }
```

一个创建交换机的请求, 形如:

- 可以把 ExchangeDeclareArguments 转成 byte[], 就得到了下列图片的结构.
- 按照 length 长度读取出 payload, 就可以把读到的二进制数据转换成 ExchangeDeclareArguments 对象.



后续请求报文格式同理, 就不再重复画了.

2) ExchangeDeleteArguments

```
1 public class ExchangeDeleteArguments extends BaseArguments implements
  Serializable {
2     private String exchangeName;
```

```
3 }
```

3) QueueDeclareArguments

```
1 public class QueueDeclareArguments extends BaseArguments implements
  Serializable {
2     private String queueName;
3     private boolean durable;
4     private boolean exclusive;
5     private boolean autoDelete;
6     private Map<String, Object> arguments;
7 }
```

4) QueueDeleteArguments

```
1 public class QueueDeleteArguments extends BaseArguments implements Serializable
  {
2     private String queueName;
3 }
```

5) QueueBindArguments

```
1 public class QueueBindArguments extends BaseArguments implements Serializable {
2     private String queueName;
3     private String exchangeName;
4     private String bindingKey;
5 }
```

6) QueueUnbindArguments

```
1 public class QueueUnbindArguments extends BaseArguments implements Serializable
  {
2     private String queueName;
```

```
3     private String exchangeName;
4 }
```

7) BasicPublishArguments

```
1 public class BasicPublishArguments extends BaseArguments implements
  Serializable {
2     private String exchangeName;
3     private String routingKey;
4     private BasicProperties basicProperties;
5     private byte[] body;
6 }
```

8) BasicConsumeArguments

```
1 public class BasicConsumeArguments extends BaseArguments implements
  Serializable {
2     private String consumeTag;
3     private String queueName;
4     private boolean autoAck;
5 }
```

9) SubScribeReturns

- 这个不是参数, 是返回值. 是服务器给消费者推送的订阅消息.
- consumerTag 其实是 channelId.
- basicProperties 和 body 共同构成了 Message.

```
1 public class SubScribeReturns extends BaseReturns implements Serializable {
2     private String consumerTag;
3     private BasicProperties basicProperties;
4     private byte[] body;
5 }
```

十二. 实现 BrokerServer

创建 BrokerServer 类

```
1 public class BrokerServer {
2     // 当前程序只考虑一个虚拟主机的情况.
3     private VirtualHost virtualHost = new VirtualHost("default-VirtualHost");
4     // key 为 channelId, value 为 channel 对应的 socket 对象.
5     private ConcurrentHashMap<String, Socket> sessions = new
        ConcurrentHashMap<>();
6
7     private ServerSocket serverSocket;
8     private ExecutorService executorService;
9     private volatile boolean runnable = true;
10 }
```

- virtualHost 表示服务器持有的虚拟主机. 队列, 交换机, 绑定, 消息都是通过虚拟主机管理.
- sessions 用来管理所有的客户端的连接. 记录每个客户端的 socket.
- serverSocket 是服务器自身的 socket
- executorService 这个线程池用来处理响应.
- runnable 这个标志位用来控制服务器的运行停止.

启动/停止服务器

- 这里就是一个单纯的 TCP 服务器, 没啥特别的.
- 实现停止操作, 主要是为了方便后续开展单元测试.

```
1 public BrokerServer(int port) throws IOException {
2     serverSocket = new ServerSocket(port);
3 }
4
5 public void start() throws IOException {
6     System.out.println("[BrokerServer] 启动完成!");
7     executorService = Executors.newCachedThreadPool();
8     try {
9         while (runnable) {
10             Socket clientSocket = serverSocket.accept();
11             executorService.submit(() -> processConnection(clientSocket));
12         }
13     } catch (SocketException e) {
```



```

14         System.out.println("[BrokerServer] 服务器关闭完成!");
15     }
16 }
17
18 public void stop() throws IOException {
19     runnable = false;
20     // 立即结束所有的线程池的任务
21     executorService.shutdownNow();
22     serverSocket.close();
23 }

```

实现处理连接

- 对于 EOFException 和 SocketException , 我们视为客户端正常断开连接。
 - 如果是客户端先 close, 后调用 DataInputStream 的 read, 则抛出 EOFException
 - 如果是先调用 DataInputStream 的 read, 后客户端调用 close, 则抛出 SocketException

```

1 private void processConnection(Socket clientSocket) {
2     try (InputStream inputStream = clientSocket.getInputStream();
3         OutputStream outputStream = clientSocket.getOutputStream()) {
4         DataInputStream dataInputStream = new DataInputStream(inputStream);
5         DataOutputStream dataOutputStream = new DataOutputStream(outputStream);
6         while (true) {
7             Request request = readRequest(dataInputStream);
8             Response response = process(request, clientSocket);
9             writeResponse(dataOutputStream, response);
10        }
11    } catch (EOFException | SocketException e) {
12        System.out.println("[BrokerServer] connection 关闭! serverIP=" +
13            clientSocket.getInetAddress().toString()
14            + ", port=" + clientSocket.getPort());
15    } catch (MqException | IOException | ClassNotFoundException e) {
16        System.out.println("[BrokerServer] connection 出现异常!");
17        e.printStackTrace();
18    } finally {
19        try {
20            clientSocket.close();
21            // 对 sessions 进行清理
22            clearClosedSession(clientSocket);
23        } catch (IOException e) {
24            e.printStackTrace();
25        }
26    }
27 }

```

```
25     }  
26 }
```

实现 readRequest

```
1 private Request readRequest(DataInputStream dataInputStream) throws  
   IOException {  
2     Request request = new Request();  
3     request.setType(dataInputStream.readInt());  
4     request.setLength(dataInputStream.readInt());  
5     byte[] payload = new byte[request.getLength()];  
6     int n = dataInputStream.read(payload);  
7     if (n != request.getLength()) {  
8         throw new IOException("读取请求数据出错!");  
9     }  
10    request.setPayload(payload);  
11    return request;  
12 }
```

实现 writeResponse

- 注意这里的 flush 操作很关键, 否则响应不一定能及时返回给客户端.

```
1 private void writeResponse(DataOutputStream dataOutputStream, Response  
   response) throws IOException {  
2     dataOutputStream.writeInt(response.getType());  
3     dataOutputStream.writeInt(response.getLength());  
4     dataOutputStream.write(response.getPayload());  
5     dataOutputStream.flush();  
6 }
```

实现处理请求

- 先把请求转换成 BaseArguments, 获取到其中的 channelId 和 rid
- 再根据不同的 type, 分别处理不同的逻辑. (主要是调用 virtualHost 中不同的方法).

- 针对消息订阅操作,则需要在存在消息的时候通过回调,把响应结果写回给对应的客户端.
- 最后构造成统一的响应.

```
1 private Response process(Request request, Socket clientSocket) throws
  MqException, IOException, ClassNotFoundException {
2     // 1. 从 request 中解析出业务请求
3     BaseArguments baseArguments = (BaseArguments)
  BinaryTool.fromBytes(request.getPayload());
4     System.out.println("[Request] rid=" + baseArguments.getRid() + ",
  channelId=" + baseArguments.getChannelId()
5         + ", type=" + request.getType() + ", length=" +
  request.getLength());
6     // 2. 根据 type 来区分业务分支.
7     boolean ok = true;
8     if (request.getType() == 0x1) {
9         // 创建 channel
10        sessions.put(baseArguments.getChannelId(), clientSocket);
11        System.out.println("[BrokerServer] 创建 channel 完成! channelId=" +
  baseArguments.getChannelId());
12    } else if (request.getType() == 0x2) {
13        // 销毁 channel
14        sessions.remove(baseArguments.getChannelId());
15        System.out.println("[BrokerServer] 销毁 channel 完成! channelId=" +
  baseArguments.getChannelId());
16    } else if (request.getType() == 0x3) {
17        // 创建交换机
18        ExchangeDeclareArguments exchangeDeclareArguments =
  (ExchangeDeclareArguments) baseArguments;
19        ok =
  virtualHost.exchangeDeclare(exchangeDeclareArguments.getExchangeName(),
  exchangeDeclareArguments.getExchangeType(),
20            exchangeDeclareArguments.isDurable(),
  exchangeDeclareArguments.isAutoDelete(),
  exchangeDeclareArguments.getArguments());
21    } else if (request.getType() == 0x4) {
22        // 删除交换机
23        ExchangeDeleteArguments exchangeDeleteArguments =
  (ExchangeDeleteArguments) baseArguments;
24        ok =
  virtualHost.exchangeDelete(exchangeDeleteArguments.getExchangeName());
25    } else if (request.getType() == 0x5) {
26        // 创建队列
27        QueueDeclareArguments queueDeclareArguments = (QueueDeclareArguments)
  baseArguments;
28        ok = virtualHost.queueDeclare(queueDeclareArguments.getQueueName(),
  queueDeclareArguments.isDurable(), queueDeclareArguments.isExclusive(),
```

```

29         queueDeclareArguments.isAutoDelete(),
queueDeclareArguments.getArguments());
30     } else if (request.getType() == 0x6) {
31         // 删除队列
32         QueueDeleteArguments queueDeleteArguments = (QueueDeleteArguments)
baseArguments;
33         ok = virtualHost.queueDelete(queueDeleteArguments.getQueueName());
34     } else if (request.getType() == 0x7) {
35         // 创建绑定
36         QueueBindArguments queueBindArguments = (QueueBindArguments)
baseArguments;
37         ok = virtualHost.queueBind(queueBindArguments.getQueueName(),
queueBindArguments.getExchangeName(), queueBindArguments.getBindingKey());
38     } else if (request.getType() == 0x8) {
39         // 解除绑定
40         QueueUnbindArguments queueUnbindArguments = (QueueUnbindArguments)
baseArguments;
41         ok = virtualHost.queueUnbind(queueUnbindArguments.getQueueName(),
queueUnbindArguments.getExchangeName());
42     } else if (request.getType() == 0x9) {
43         // 发送消息
44         BasicPublishArguments basicPublishArguments = (BasicPublishArguments)
baseArguments;
45         ok = virtualHost.basicPublish(basicPublishArguments.getExchangeName(),
basicPublishArguments.getRoutingKey(),
46             basicPublishArguments.getBasicProperties(),
basicPublishArguments.getBody());
47     } else if (request.getType() == 0xa) {
48         // 订阅消息
49         BasicConsumeArguments basicConsumeArguments = (BasicConsumeArguments)
baseArguments;
50         // 创建个回调，用来把消费的数据转发回客户端。
51         ok = virtualHost.basicConsume(basicConsumeArguments.getConsumeTag(),
basicConsumeArguments.getQueueName(),
52             basicConsumeArguments.isAutoAck(), new Consumer() {
53                 @Override
54                 public void handleDelivery(String consumerTag,
BasicProperties properties, byte[] body) throws MqException, IOException {
55                     // 1. 根据 channelId 找到对应的 socket
56                     Socket clientSocket =
sessions.get(basicConsumeArguments.getChannelId());
57                     if (clientSocket == null || clientSocket.isClosed()) {
58                         throw new MqException("[BrokerServer] 订阅消息的客户
端已经关闭!");
59                     }
60                     // 2. 构造响应数据

```

```

61         SubSubscribeReturns subSubscribeReturns = new
SubSubscribeReturns();
62
        subSubscribeReturns.setChannelId(basicConsumeArguments.getChannelId());
63         subSubscribeReturns.setConsumerTag(consumerTag);
64         subSubscribeReturns.setBasicProperties(properties);
65         subSubscribeReturns.setBody(body);
66         byte[] payload = BinaryTool.toBytes(subSubscribeReturns);
67         // 3. 写入到对应 socket 中
68         Response response = new Response();
69         response.setType(0xc);
70         response.setLength(payload.length);
71         response.setPayload(payload);
72         // 此处不应该关闭 DataOutputStream, 关闭这个会导致内部持有
        的 clientSocket.getOutputStream 被关闭.
73         DataOutputStream dataOutputStream = new
DataOutputStream(clientSocket.getOutputStream());
74         writeResponse(dataOutputStream, response);
75     }
76     });
77     } else if (request.getType() == 0xb) {
78         // 确认 ack
79         BasicAckArguments basicAckArguments = (BasicAckArguments)
baseArguments;
80         ok = virtualHost.basicAck(basicAckArguments.getQueueName(),
baseAckArguments.getMessageId());
81     } else {
82         throw new MqException("[BrokerServer] 未知的请求 type ! type=" +
request.getType());
83     }
84     // 3. 构造响应.
85     BaseReturns baseReturns = new BaseReturns();
86     baseReturns.setRid(baseArguments.getRid());
87     baseReturns.setChannelId(baseArguments.getChannelId());
88     baseReturns.setOk(ok);
89
90     byte[] payload = BinaryTool.toBytes(baseReturns);
91     Response response = new Response();
92     response.setType(request.getType());
93     response.setLength(payload.length);
94     response.setPayload(payload);
95     System.out.println("[Response] rid=" + baseReturns.getRid() + ",
channelId=" + baseReturns.getChannelId()
96         + ", type=" + response.getType() + ", length=" +
response.getLength());
97     return response;
98 }

```

实现 clearClosedSession

- 如果客户端只关闭了 Connection, 没关闭 Connection 中包含的 Channel, 也没关系, 在这里统一进行清理.
- 注意迭代器失效问题.

```
1 private void clearClosedSession(Socket clientSocket) {
2     // 这里不要在同一循环中, 同时进行遍历 + 删除操作. 否则可能有迭代器失效问题.
3     // 拆成两个循环来处理是更合适的.
4     List<String> toDeleteChannelId = new ArrayList<>();
5     for (Map.Entry<String, Socket> entry : sessions.entrySet()) {
6         if (entry.getValue() == clientSocket) {
7             toDeleteChannelId.add(entry.getKey());
8         }
9     }
10    for (String channelId : toDeleteChannelId) {
11        sessions.remove(channelId);
12    }
13    System.out.println("[BrokerServer] 清理 session 完成! 被清理的 channelId=" +
14        toDeleteChannelId);
15 }
```

十三. 实现客户端

创建包 mqclient

创建 ConnectionFactory

用来创建连接的工厂类.

- 当前没有实现用户认证和多虚拟主机, 用户名密码可以暂时先不要.

```
1 public class ConnectionFactory {
2     // BrokerServer 的 ip 和 port
3     private String host;
4     private int port;
5     // 这几个部分暂时不加.
6     // private String virtualHost;
```

```

7 // private String username;
8 // private String password;
9
10 // 建立一个 tcp 连接
11 public Connection newConnection() throws IOException {
12     Connection connection = new Connection(host, port);
13     return connection;
14 }
15 }

```

Connection 和 Channel 的定义

一个客户端可以创建多个 Connection.

一个 Connection 对应一个 socket, 一个 TCP 连接.

一个 Connection 可以包含多个 Channel

1) Connection 的定义

```

1 public class Connection {
2     private Socket socket;
3     private InputStream inputStream;
4     private OutputStream outputStream;
5     private DataInputStream dataInputStream;
6     private DataOutputStream dataOutputStream;
7     // 记录当前 Connection 包含的 Channel
8     private ConcurrentHashMap<String, Channel> channelMap = new
ConcurrentHashMap<>();
9     // 执行消费消息回调的线程池
10    private ExecutorService callbackPool = Executors.newFixedThreadPool(4);
11 }

```

- Socket 是客户端持有的套接字. InputStream OutputStream DataInputStream DataOutputStream 均为 socket 通信的接口.
- channelMap 用来管理该连接中所有的 Channel.
- callbackPool 是用来在客户端这边执行用户回调的线程池.

2) Channel 的定义


```

1 public class Channel {
2     private String channelId;
3     private Connection connection;
4     // key 为 rid, 即 requestId / responseId.
5     private ConcurrentHashMap<String, BaseReturns> baseReturnsMap = new
    ConcurrentHashMap<>();
6     // 订阅消息的回调
7     private Consumer consumer = null;
8
9     public Channel(String channelId, Connection connection) {
10         this.channelId = channelId;
11         this.connection = connection;
12     }
13 }

```

- channelId 为 channel 的身份标识, 使用 UUID 标识.
- Connection 为 channel 对应的连接.
- baseReturnsMap 用来保存响应的返回值. 放到这个哈希表中方便和请求匹配.
- consumer 为消费者的回调(用户注册的). 对于消息响应, 应该调用这个回调处理消息.

封装请求响应读写操作

在 Connection 中, 实现下列方法

```

1 // 读取响应应该在另外一个单独的线程中完成.
2 public void writeRequest(Request request) throws IOException {
3     dataOutputStream.writeInt(request.getType());
4     dataOutputStream.writeInt(request.getLength());
5     dataOutputStream.write(request.getPayload());
6     dataOutputStream.flush();
7     System.out.println("[Connection] 发送请求! type=" + request.getType() + ",
    length=" + request.getLength());
8 }

```

```

1 public Response readResponse() throws IOException {
2     Response response = new Response();
3     response.setType(dataInputStream.readInt());
4     response.setLength(dataInputStream.readInt());
5     byte[] payload = new byte[response.getLength()];

```



```

6      int n = dataInputStream.read(payload);
7      if (n != response.getLength()) {
8          throw new IOException("读取到的响应数据不完整!");
9      }
10     response.setPayload(payload);
11     System.out.println("[Connection] 收到响应! type=" + response.getType() + ",
length=" + response.getLength());
12     return response;
13 }

```

创建 channel

在 Connection 中, 定义下列方法来创建一个 channel

```

1 public Channel createChannel() throws IOException {
2     // 使用 UUID 生产 channelId, 以 C- 开头
3     String channelId = "C-" + UUID.randomUUID().toString();
4     Channel channel = new Channel(channelId, this);
5     // 这里需要先把 channel 键值对放到 Map 中. 否则后续 createChannel 的阻塞等待就等
    不到结果了
6     channelMap.put(channelId, channel);
7     boolean ok = channel.createChannel();
8     if (!ok) {
9         // 服务器返回创建 channel 失败!
10        // 把 channelId 删除掉即可
11        channelMap.remove(channelId);
12        return null;
13    }
14    return channel;
15 }

```

发送请求

通过 Channel 提供请求的发送操作.

1) 创建 channel

```

1 public boolean createChannel() throws IOException {

```

```

2     BaseArguments baseArguments = new BaseArguments();
3     baseArguments.setRid(generateRid());
4     baseArguments.setChannelId(channelId);
5     byte[] payload = BinaryTool.toBytes(baseArguments);
6     Request request = new Request();
7     request.setType(0x1);
8     request.setLength(payload.length);
9     request.setPayload(payload);
10    connection.writeRequest(request);
11    // 阻塞等待服务器的响应
12    BaseReturns baseReturns = waitResult(baseArguments.getRid());
13    return baseReturns.isOk();
14 }

```

generateRid 的实现

```

1 private String generateRid() {
2     return "R-" + UUID.randomUUID().toString();
3 }

```

waitResult 的实现

- 由于服务器的响应是异步的. 此处通过 waitResult 实现同步等待的效果.

```

1 private BaseReturns waitResult(String rid){
2     BaseReturns baseReturns = null;
3     while ((baseReturns = baseReturnsMap.get(rid)) == null) {
4         synchronized (this) {
5             try {
6                 wait();
7             } catch (InterruptedException e) {
8                 // 如果 wait 被提前唤醒, 也应该继续循环.
9                 // 所以这里啥都不干, 但是 try 需要放到 while 内部.
10                e.printStackTrace();
11            }
12        }
13    }
14    return baseReturns;
15 }

```

2) 关闭 channel

```
1 public boolean close() throws IOException {
2     // 删除服务器上的 channel。如果不显式调用，也没关系。服务器会在 Connection 断开的时候统一回收。
3     BaseArguments baseArguments = new BaseArguments();
4     baseArguments.setRid(generateRid());
5     baseArguments.setChannelId(channelId);
6     byte[] payload = BinaryTool.toBytes(baseArguments);
7     Request request = new Request();
8     request.setType(0x2);
9     request.setLength(payload.length);
10    request.setPayload(payload);
11    connection.writeRequest(request);
12    // 阻塞等待服务器的响应
13    BaseReturns baseReturns = waitResult(baseArguments.getRid());
14    return baseReturns.isOk();
15 }
```

3) 创建交换机

```
1 public boolean exchangeDeclare(String exchangeName, ExchangeType exchangeType,
2     boolean durable, boolean autoDelete,
3     Map<String, Object> arguments) throws IOException {
4     ExchangeDeclareArguments exchangeDeclareArguments = new
5     ExchangeDeclareArguments();
6     exchangeDeclareArguments.setRid(generateRid());
7     exchangeDeclareArguments.setChannelId(channelId);
8     exchangeDeclareArguments.setExchangeName(exchangeName);
9     exchangeDeclareArguments.setExchangeType(exchangeType);
10    exchangeDeclareArguments.setDurable(durable);
11    exchangeDeclareArguments.setAutoDelete(autoDelete);
12    exchangeDeclareArguments.setArguments(arguments);
13    byte[] payload = BinaryTool.toBytes(exchangeDeclareArguments);
14
15    Request request = new Request();
16    request.setType(0x3);
17    request.setLength(payload.length);
18    request.setPayload(payload);
19    connection.writeRequest(request);
20 }
```

```
18
19 // 阻塞等待服务器的响应
20 BaseReturns baseReturns = waitResult(exchangeDeclareArguments.getRid());
21 return baseReturns.isOk();
22 }
```

4) 删除交换机

```
1 public boolean exchangeDelete(String exchangeName) throws IOException {
2     ExchangeDeleteArguments exchangeDeleteArguments = new
    ExchangeDeleteArguments();
3     exchangeDeleteArguments.setRid(generateRid());
4     exchangeDeleteArguments.setChannelId(channelId);
5     exchangeDeleteArguments.setExchangeName(exchangeName);
6     byte[] payload = BinaryTool.toBytes(exchangeDeleteArguments);
7
8     Request request = new Request();
9     request.setType(0x4);
10    request.setLength(payload.length);
11    request.setPayload(payload);
12    connection.writeRequest(request);
13
14    // 阻塞等待服务器的响应
15    BaseReturns baseReturns = waitResult(exchangeDeleteArguments.getRid());
16    return baseReturns.isOk();
17 }
```

5) 创建队列

```
1 public boolean queueDeclare(String queueName, boolean durable, boolean
    exclusive, boolean autoDelete,
2     Map<String, Object> arguments) throws IOException {
3     QueueDeclareArguments queueDeclareArguments = new QueueDeclareArguments();
4     queueDeclareArguments.setRid(generateRid());
5     queueDeclareArguments.setChannelId(channelId);
6     queueDeclareArguments.setQueueName(queueName);
7     queueDeclareArguments.setDurable(durable);
8     queueDeclareArguments.setExclusive(exclusive);
9     queueDeclareArguments.setAutoDelete(autoDelete);
```

```

10 queueDeclareArguments.setArguments(arguments);
11 byte[] payload = BinaryTool.toBytes(queueDeclareArguments);
12
13 Request request = new Request();
14 request.setType(0x5);
15 request.setLength(payload.length);
16 request.setPayload(payload);
17 connection.writeRequest(request);
18
19 // 阻塞等待服务器的响应
20 BaseReturns baseReturns = waitResult(queueDeclareArguments.getRid());
21 return baseReturns.isOk();
22 }

```

6) 删除队列

```

1 public boolean queueDelete(String queueName) throws IOException {
2     QueueDeleteArguments queueDeleteArguments = new QueueDeleteArguments();
3     queueDeleteArguments.setRid(generateRid());
4     queueDeleteArguments.setChannelId(channelId);
5     queueDeleteArguments.setQueueName(queueName);
6     byte[] payload = BinaryTool.toBytes(queueDeleteArguments);
7
8     Request request = new Request();
9     request.setType(0x6);
10    request.setLength(payload.length);
11    request.setPayload(payload);
12    connection.writeRequest(request);
13
14    // 阻塞等待服务器的响应
15    BaseReturns baseReturns = waitResult(queueDeleteArguments.getRid());
16    return baseReturns.isOk();
17 }

```

7) 创建绑定

```

1 // 对于直接交换机和 fanout 交换机, bindingKey 不生效. 直接设为 "" 即可
2 public boolean queueBind(String queueName, String exchangeName) throws
    IOException {

```

```

3     return queueBind(queueName, exchangeName, "");
4 }
5
6 public boolean queueBind(String queueName, String exchangeName, String
    bindingKey) throws IOException {
7     QueueBindArguments queueBindArguments = new QueueBindArguments();
8     queueBindArguments.setRid(generateRid());
9     queueBindArguments.setChannelId(channelId);
10    queueBindArguments.setQueueName(queueName);
11    queueBindArguments.setExchangeName(exchangeName);
12    queueBindArguments.setBindingKey(bindingKey);
13    byte[] payload = BinaryTool.toBytes(queueBindArguments);
14
15    Request request = new Request();
16    request.setType(0x7);
17    request.setLength(payload.length);
18    request.setPayload(payload);
19    connection.writeRequest(request);
20
21    // 阻塞等待服务器的响应
22    BaseReturns baseReturns = waitResult(queueBindArguments.getRid());
23    return baseReturns.isOk();
24 }

```

8) 删除绑定

```

1 public boolean queueUnbind(String queueName, String exchangeName) throws
    IOException {
2     QueueUnbindArguments queueUnbindArguments = new QueueUnbindArguments();
3     queueUnbindArguments.setRid(generateRid());
4     queueUnbindArguments.setChannelId(channelId);
5     queueUnbindArguments.setQueueName(queueName);
6     queueUnbindArguments.setExchangeName(exchangeName);
7     byte[] payload = BinaryTool.toBytes(queueUnbindArguments);
8
9     Request request = new Request();
10    request.setType(0x8);
11    request.setLength(payload.length);
12    request.setPayload(payload);
13    connection.writeRequest(request);
14
15    // 阻塞等待服务器的响应
16    BaseReturns baseReturns = waitResult(queueUnbindArguments.getRid());

```

```
17     return baseReturns.isOk();
18 }
```

9) 发送消息

```
1 public boolean basicPublish(String exchangeName, String routingKey,
    BasicProperties basicProperties, byte[] body) throws IOException {
2     BasicPublishArguments basicPublishArguments = new BasicPublishArguments();
3     basicPublishArguments.setRid(generateRid());
4     basicPublishArguments.setChannelId(channelId);
5     basicPublishArguments.setExchangeName(exchangeName);
6     basicPublishArguments.setRoutingKey(routingKey);
7     basicPublishArguments.setBasicProperties(basicProperties);
8     basicPublishArguments.setBody(body);
9     byte[] payload = BinaryTool.toBytes(basicPublishArguments);
10
11     Request request = new Request();
12     request.setType(0x9);
13     request.setLength(payload.length);
14     request.setPayload(payload);
15     connection.writeRequest(request);
16
17     // 阻塞等待服务器的响应
18     BaseReturns baseReturns = waitResult(basicPublishArguments.getRid());
19     return baseReturns.isOk();
20 }
```

10) 订阅消息

```
1 public boolean basicConsume(String queueName, boolean autoAck, Consumer
    consumer) throws IOException, MqException {
2     BasicConsumeArguments basicConsumeArguments = new BasicConsumeArguments();
3     basicConsumeArguments.setRid(generateRid());
4     basicConsumeArguments.setChannelId(channelId);
5     basicConsumeArguments.setQueueName(queueName);
6     basicConsumeArguments.setAutoAck(autoAck);
7     basicConsumeArguments.setConsumeTag(channelId);
8     byte[] payload = BinaryTool.toBytes(basicConsumeArguments);
9 }
```

```

10 Request request = new Request();
11 request.setType(0xa);
12 request.setLength(payload.length);
13 request.setPayload(payload);
14 connection.writeRequest(request);
15
16 // 阻塞等待服务器的响应
17 BaseReturns baseReturns = waitResult(basicConsumeArguments.getRid());
18 if (baseReturns.isOk()) {
19     // 设置回调
20     if (this.consumer != null) {
21         throw new MqException("该 channel 已经设置过消费回调，不能重复设置!");
22     }
23     this.consumer = consumer;
24 }
25 return baseReturns.isOk();
26 }

```

11) 确认消息

```

1 public boolean basicAck(String queueName, String messageId) throws IOException
2 {
3     BasicAckArguments basicAckArguments = new BasicAckArguments();
4     basicAckArguments.setRid(generateRid());
5     basicAckArguments.setChannelId(channelId);
6     basicAckArguments.setQueueName(queueName);
7     basicAckArguments.setMessageId(messageId);
8     byte[] payload = BinaryTool.toBytes(basicAckArguments);
9
10    Request request = new Request();
11    request.setType(0xb);
12    request.setLength(payload.length);
13    request.setPayload(payload);
14    connection.writeRequest(request);
15    // 阻塞等待服务器的响应
16    BaseReturns baseReturns = waitResult(basicAckArguments.getRid());
17    return baseReturns.isOk();
18 }

```

小结

上述发送请求的操作, 逻辑基本一致. 构造参数 + 构造请求 + 发送 + 等待结果.

处理响应

1) 创建扫描线程

创建一个扫描线程, 用来不停的读取 socket 中的响应数据.

注意: 一个 Connection 中可能包含多个 channel, 需要把响应分别放到对应的 channel 中.

```
1 public Connection(String host, int port) throws IOException {
2     socket = new Socket(host, port);
3     inputStream = socket.getInputStream();
4     outputStream = socket.getOutputStream();
5     dataInputStream = new DataInputStream(inputStream);
6     dataOutputStream = new DataOutputStream(outputStream);
7
8     // 创建一个读响应的线程
9     Thread t = new Thread(() -> {
10         try {
11             while (!socket.isClosed()) {
12                 Response response = readResponse();
13                 dispatchResponse(response);
14             }
15         } catch (SocketException e) {
16             // 连接断开, 忽略该异常.
17             // System.out.println("[Connection] 连接断开!");
18         } catch (IOException | ClassNotFoundException | MqException e) {
19             System.out.println("[Connection] 连接出现异常!");
20             e.printStackTrace();
21         }
22     });
23     t.start();
24 }
```

2) 实现响应的分发

给 Connection 创建 dispatchResponse 方法.

- 针对服务器返回的控制响应和消息响应, 分别处理.
 - 如果是订阅数据, 则调用 channel 中的回调.

- 如果是控制消息, 直接放到结果集合中.

```
1 private void dispatchResponse(Response response) throws IOException,
  ClassNotFoundException, MqException {
2     if (response.getType() == 0xc) {
3         // 1. 解析到服务器返回的订阅数据
4         SubScribeReturns subScribeReturns = (SubScribeReturns)
          BinaryTool.fromBytes(response.getPayload());
5         // 2. 获取到 channel
6         Channel channel = channelMap.get(subScribeReturns.getChannelId());
7         if (channel == null) {
8             throw new MqException("该消息对应的 channel 不存在! channelId=" +
          subScribeReturns.getChannelId());
9         }
10        // 3. 执行 channel 中对应的回调.
11        callbackPool.submit(() -> {
12            try {
13
14                channel.getConsumer().handleDelivery(subScribeReturns.getConsumerTag(),
          subScribeReturns.getBasicProperties(),
          subScribeReturns.getBody());
15            } catch (MqException | IOException e) {
16                e.printStackTrace();
17            }
18        });
19    } else {
20        // 1. 拿到服务器返回的控制消息
21        BaseReturns baseReturns = (BaseReturns)
          BinaryTool.fromBytes(response.getPayload());
22        // System.out.printf("[Connection] 收到响应: type=0x%x, channelId=%s,
          ok=%b\n", response.getType(),
23            // baseReturns.getChannelId(), baseReturns.isOk());
24        // 2. 找到对应 Channel
25        Channel channel = channelMap.get(baseReturns.getChannelId());
26        if (channel == null) {
27            // 这个是小问题, 不要抛异常
28            System.out.println("[Connection] channel 不存在! channelId=" +
          baseReturns.getChannelId());
29            return;
30        }
31        // 3. 把响应放到对应的 Channel 的 map 中.
32        channel.putReturns(baseReturns);
33    }
34 }
```

3) 实现 channel.putReturns

把响应放到响应的 hash 表中, 同时唤醒等待响应的线程去消费.

```
1 public void putReturns(BaseReturns baseReturns) {
2     baseReturnsMap.put(baseReturns.getRid(), baseReturns);
3     synchronized (this) {
4         // 这里要唤醒所有等待的线程, 不能只唤醒一个.
5         notifyAll();
6     }
7 }
```

关闭 Connection

给 Connection 实现 close 方法

```
1 public void close() {
2     try {
3         callbackPool.shutdown();
4         channelMap = null;
5         inputStream.close();
6         outputStream.close();
7         socket.close();
8     } catch (IOException e) {
9         e.printStackTrace();
10    }
11 }
```

测试客户端-服务器

创建 MqClientTests

```
1 public class MqClientTests {
2     private BrokerServer brokerServer = null;
3     private Thread t = null;
4
5     private ConnectionFactory factory = null;
```

```

6
7     @BeforeEach
8     public void setUp() throws IOException {
9         JavaMessageQueueApplication.ac =
SpringApplication.run(JavaMessageQueueApplication.class);
10        t = new Thread(() -> {
11            try {
12                brokerServer = new BrokerServer(9090);
13                brokerServer.start();
14            } catch (IOException e) {
15                e.printStackTrace();
16            }
17        });
18        t.start();
19
20        factory = new ConnectionFactory();
21        factory.setHost("127.0.0.1");
22        factory.setPort(9090);
23    }
24
25    @AfterEach
26    public void tearDown() throws IOException, InterruptedException {
27        // 结束服务器
28        brokerServer.stop();
29        // 等待线程结束
30        t.join();
31        // 关闭 Spring 服务器
32        JavaMessageQueueApplication.ac.close();
33        // 删除服务器的数据文件
34        File dbFile = new File("meta.db");
35        dbFile.delete();
36        // 删除数据文件
37        File dataFile = new File("./data");
38        FileUtils.deleteDirectory(dataFile);
39
40        factory = null;
41    }
42 }

```

编写测试用例

```

1 @Test
2 public void testConnection() throws IOException {
3     Connection connection = factory.newConnection();
4     Assertions.assertNotNull(connection);

```

```
5 }
6
7 @Test
8 public void testChannel() throws IOException {
9     Connection connection = factory.newConnection();
10    Assertions.assertNotNull(connection);
11    Channel channel = connection.createChannel();
12    Assertions.assertNotNull(channel);
13 }
14
15 @Test
16 public void testExchange() throws IOException, InterruptedException {
17     Connection connection = factory.newConnection();
18     Assertions.assertNotNull(connection);
19     Channel channel = connection.createChannel();
20     Assertions.assertNotNull(channel);
21
22     boolean ok = channel.exchangeDeclare("testExchange", ExchangeType.DIRECT,
true, false, null);
23     Assertions.assertTrue(ok);
24     ok = channel.exchangeDelete("testExchange");
25     Assertions.assertTrue(ok);
26
27     channel.close();
28     connection.close();
29 }
30
31 @Test
32 public void testQueue() throws IOException {
33     Connection connection = factory.newConnection();
34     Assertions.assertNotNull(connection);
35     Channel channel = connection.createChannel();
36     Assertions.assertNotNull(channel);
37
38     boolean ok = channel.queueDeclare("testQueue", true, false, false, null);
39     Assertions.assertTrue(ok);
40     ok = channel.queueDelete("testQueue");
41     Assertions.assertTrue(ok);
42
43     channel.close();
44     connection.close();
45 }
46
47 @Test
48 public void testBind() throws IOException {
49     Connection connection = factory.newConnection();
50     Assertions.assertNotNull(connection);
```

```
51     Channel channel = connection.createChannel();
52     Assertions.assertNotNull(channel);
53
54     boolean ok = channel.exchangeDeclare("testExchange", ExchangeType.DIRECT,
true, false, null);
55     Assertions.assertTrue(ok);
56     ok = channel.queueDeclare("testQueue", true, false, false, null);
57     Assertions.assertTrue(ok);
58
59     ok = channel.queueBind("testQueue", "testExchange");
60     Assertions.assertTrue(ok);
61
62     ok = channel.queueUnbind("testQueue", "testExchange");
63     Assertions.assertTrue(ok);
64
65     channel.close();
66     connection.close();
67 }
68
69 @Test
70 public void testMessageDirect() throws IOException, MqException,
InterruptedException {
71     Connection connection = factory.newConnection();
72     Assertions.assertNotNull(connection);
73     Channel channel = connection.createChannel();
74     Assertions.assertNotNull(channel);
75
76     boolean ok = channel.exchangeDeclare("testExchange", ExchangeType.DIRECT,
true, false, null);
77     Assertions.assertTrue(ok);
78     ok = channel.queueDeclare("testQueue", true, false, false, null);
79     Assertions.assertTrue(ok);
80
81     byte[] requestBody = "hello".getBytes();
82     // DIRECT 模式, routingKey 就是队列名字
83     // 发送的时候 basicProperties 可以是空着的。服务器会进行构造。订阅者收到的消息则是
带有完整 basicProperties 的。
84     ok = channel.basicPublish("testExchange", "testQueue", null, requestBody);
85     Assertions.assertTrue(ok);
86
87     ok = channel.basicConsume("testQueue", true, new Consumer() {
88         @Override
89         public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] responseBody) {
90             System.out.println("[消费数据] 开始!");
91             System.out.println("consumerTag=" + consumerTag);
92             System.out.println("properties=" + properties);
```

```

93         String bodyString = new String(responseBody, 0,
responseBody.length);
94         System.out.println("body=" + bodyString);
95
96         Assertions.assertEquals(requestBody, responseBody);
97     }
98 });
99 Assertions.assertTrue(ok);
100 // 等待数据消费完.
101 Thread.sleep(500);
102
103 channel.close();
104 connection.close();
105 }
106
107 @Test
108 public void testMessageFanout() throws IOException, MqException,
InterruptedException {
109     Connection connection = factory.newConnection();
110     Assertions.assertNotNull(connection);
111     Channel channel1 = connection.createChannel();
112     Assertions.assertNotNull(channel1);
113
114     boolean ok = channel1.exchangeDeclare("testExchange", ExchangeType.FANOUT,
true, false, null);
115     Assertions.assertTrue(ok);
116     ok = channel1.queueDeclare("testQueue1", true, false, false, null);
117     Assertions.assertTrue(ok);
118     ok = channel1.queueDeclare("testQueue2", true, false, false, null);
119     Assertions.assertTrue(ok);
120     ok = channel1.queueBind("testQueue1", "testExchange");
121     Assertions.assertTrue(ok);
122     ok = channel1.queueBind("testQueue2", "testExchange");
123     Assertions.assertTrue(ok);
124
125     byte[] requestBody = "hello".getBytes();
126     // FANOUT 模式, routingKey 不需要
127     ok = channel1.basicPublish("testExchange", "", null, requestBody);
128     Assertions.assertTrue(ok);
129
130     ok = channel1.basicConsume("testQueue1", true, new Consumer() {
131         @Override
132         public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] responseBody) throws MqException, IOException {
133             System.out.println("consumerTag=" + consumerTag);
134             System.out.println("properties=" + properties);

```

```
135         String bodyString = new String(responseBody, 0,
responseBody.length);
136         System.out.println("body=" + bodyString);
137
138         Assertions.assertEquals(requestBody, responseBody);
139     }
140 });
141 Assertions.assertTrue(ok);
142
143 Channel channel2 = connection.createChannel();
144 Assertions.assertNotNull(channel1);
145
146 ok = channel2.basicConsume("testQueue2", true, new Consumer() {
147     @Override
148     public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] responseBody) throws MqException, IOException {
149         System.out.println("consumerTag=" + consumerTag);
150         System.out.println("properties=" + properties);
151         String bodyString = new String(responseBody, 0,
responseBody.length);
152         System.out.println("body=" + bodyString);
153
154         Assertions.assertEquals(requestBody, responseBody);
155     }
156 });
157 Assertions.assertTrue(ok);
158
159 Thread.sleep(1000);
160 channel1.close();
161 channel2.close();
162 connection.close();
163 }
164
165 @Test
166 public void testMessageTopic() throws IOException, MqException,
InterruptedException {
167     Connection connection = factory.newConnection();
168     Assertions.assertNotNull(connection);
169     Channel channel = connection.createChannel();
170     Assertions.assertNotNull(channel);
171
172     boolean ok = channel.exchangeDeclare("testExchange", ExchangeType.TOPIC,
true, false, null);
173     Assertions.assertTrue(ok);
174     ok = channel.queueDeclare("testQueue", true, false, false, null);
175     Assertions.assertTrue(ok);
176     ok = channel.queueBind("testQueue", "testExchange", "aaa.#");
```



```

177
178     byte[] requestBody = "hello".getBytes();
179     ok = channel.basicPublish("testExchange", "aaa.bbb.ccc", null,
requestBody);
180     Assertions.assertTrue(ok);
181
182     ok = channel.basicConsume("testQueue", true, new Consumer() {
183         @Override
184         public void handleDelivery(String consumerTag, BasicProperties
properties, byte[] responseBody) {
185             System.out.println("[消费数据] 开始!");
186             System.out.println("consumerTag=" + consumerTag);
187             System.out.println("properties=" + properties);
188             String bodyString = new String(responseBody, 0,
responseBody.length);
189             System.out.println("body=" + bodyString);
190
191             Assertions.assertEquals(requestBody, responseBody);
192         }
193     });
194     Assertions.assertTrue(ok);
195     // 等待数据消费完.
196     Thread.sleep(500);
197
198     channel.close();
199     connection.close();
200 }

```

十四. 案例: 基于 MQ 的生产者消费者模型

生产者:

```

1 public class DemoProducer {
2     public static void main(String[] args) throws IOException,
InterruptedException {
3         System.out.println("启动生产者!");
4         ConnectionFactory factory = new ConnectionFactory();
5         factory.setHost("127.0.0.1");
6         factory.setPort(9090);
7         Connection connection = factory.newConnection();
8         Channel channel = connection.createChannel();
9

```

```

10     channel.exchangeDeclare("testExchange", ExchangeType.DIRECT, true,
    false, null);
11     channel.queueDeclare("testQueue", true, false, false, null);
12
13     byte[] body = "hello".getBytes();
14     boolean ok = channel.basicPublish("testExchange", "testQueue", null,
    body);
15     System.out.println("投递消息完成! ok=" + ok);
16
17     Thread.sleep(500);
18     channel.close();
19     connection.close();
20 }
21 }

```

消费者:

```

1 public class DemoConsumer {
2     public static void main(String[] args) throws IOException, MqException,
    InterruptedException {
3         System.out.println("启动消费者!");
4         ConnectionFactory factory = new ConnectionFactory();
5         factory.setHost("127.0.0.1");
6         factory.setPort(9090);
7         Connection connection = factory.newConnection();
8         Channel channel = connection.createChannel();
9
10        channel.exchangeDeclare("testExchange", ExchangeType.DIRECT, true,
    false, null);
11        channel.queueDeclare("testQueue", true, false, false, null);
12
13        channel.basicConsume("testQueue", true, new Consumer() {
14            @Override
15            public void handleDelivery(String consumerTag, BasicProperties
    properties, byte[] body) {
16                System.out.println("[消费数据] 开始!");
17                System.out.println("consumerTag=" + consumerTag);
18                System.out.println("properties=" + properties);
19                String bodyString = new String(body, 0, body.length);
20                System.out.println("body=" + bodyString);
21                System.out.println("[消费数据] 完毕!");
22            }
23        });
24

```

```
25         while (true) {  
26             Thread.sleep(500);  
27         }  
28     }  
29 }
```

十五. 扩展功能

- 虚拟主机管理
- 用户管理/用户认证
- 交换机/队列 的独占模式和自动删除.
- 发送方确认(broker 给生产者的确认应答)
- 拒绝应答 (nack)
- 死信队列
- 管理接口
- 管理页面